

GOTO
COPENHAGEN 2025

SEPT 29 - OCT 3

Lag to Lightning: Confident, Automated Changes at Scale

Learn how to safely deploy automated changes
across thousands of repos in hours



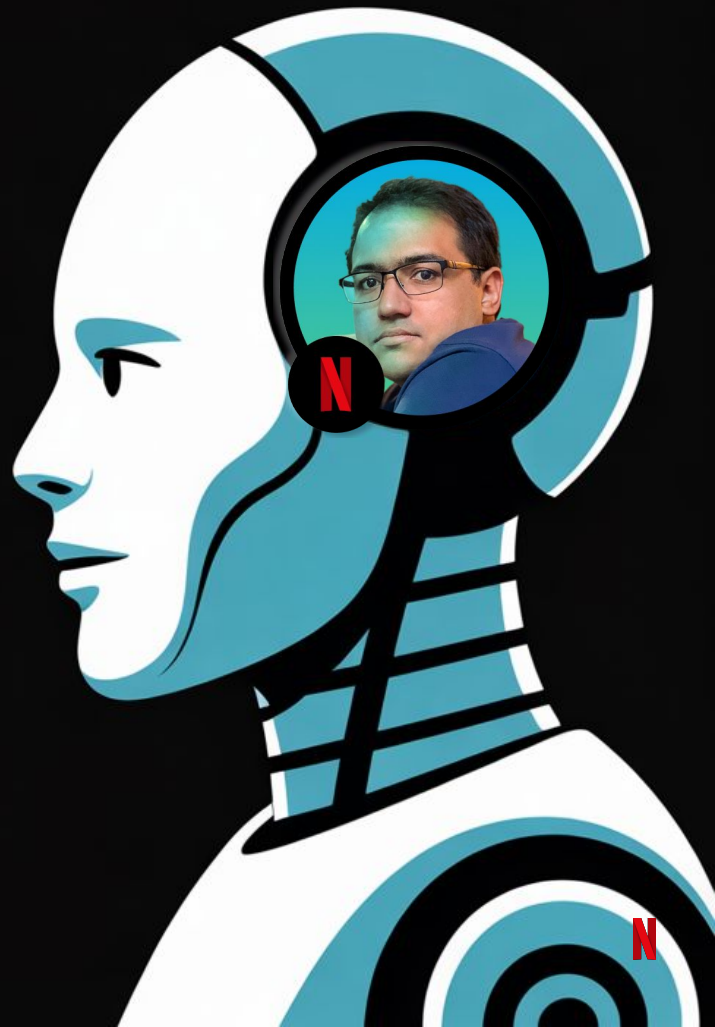
**Roberto Perez Alcolea &
Aubrey Chipman**

Software engineers at Netflix

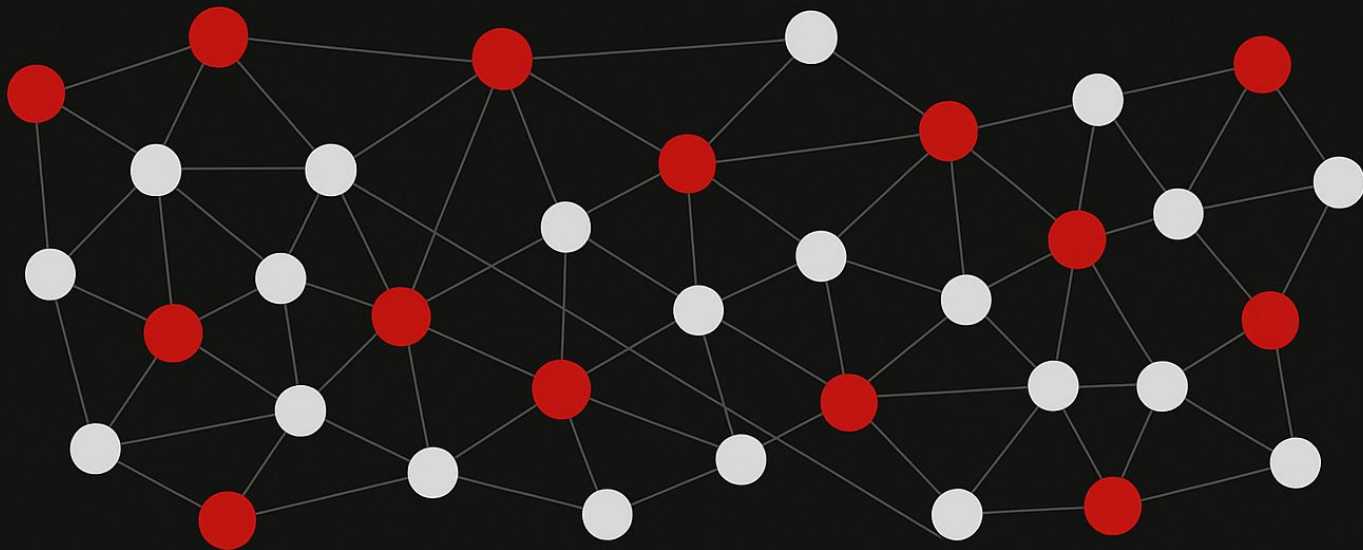
NETFLIX

**SPOILER
ALERT,
THIS IS NOT
AN AI TALK**

Image generated with <https://ideogram.ai/>



Our focus: binary dependency management in a multi-repository world



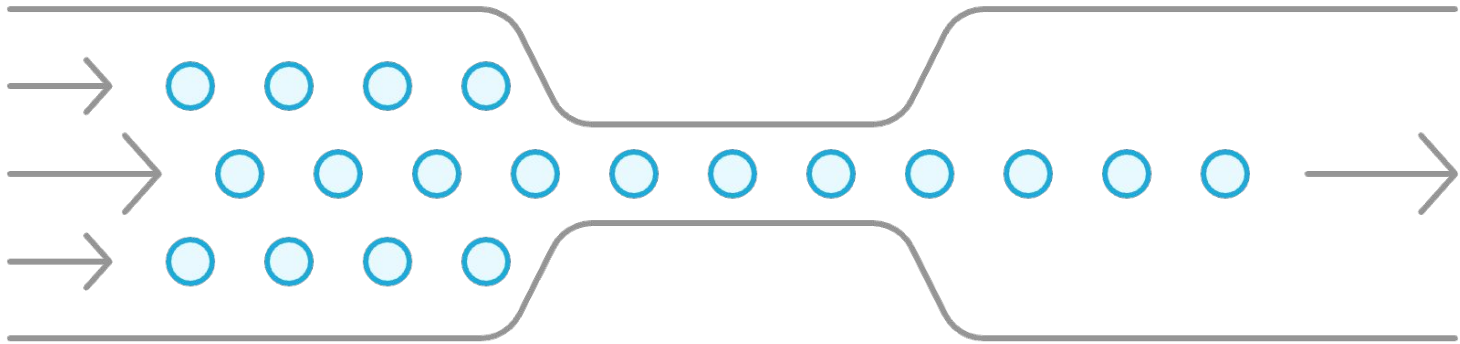
● Services ● Library

NETFLIX

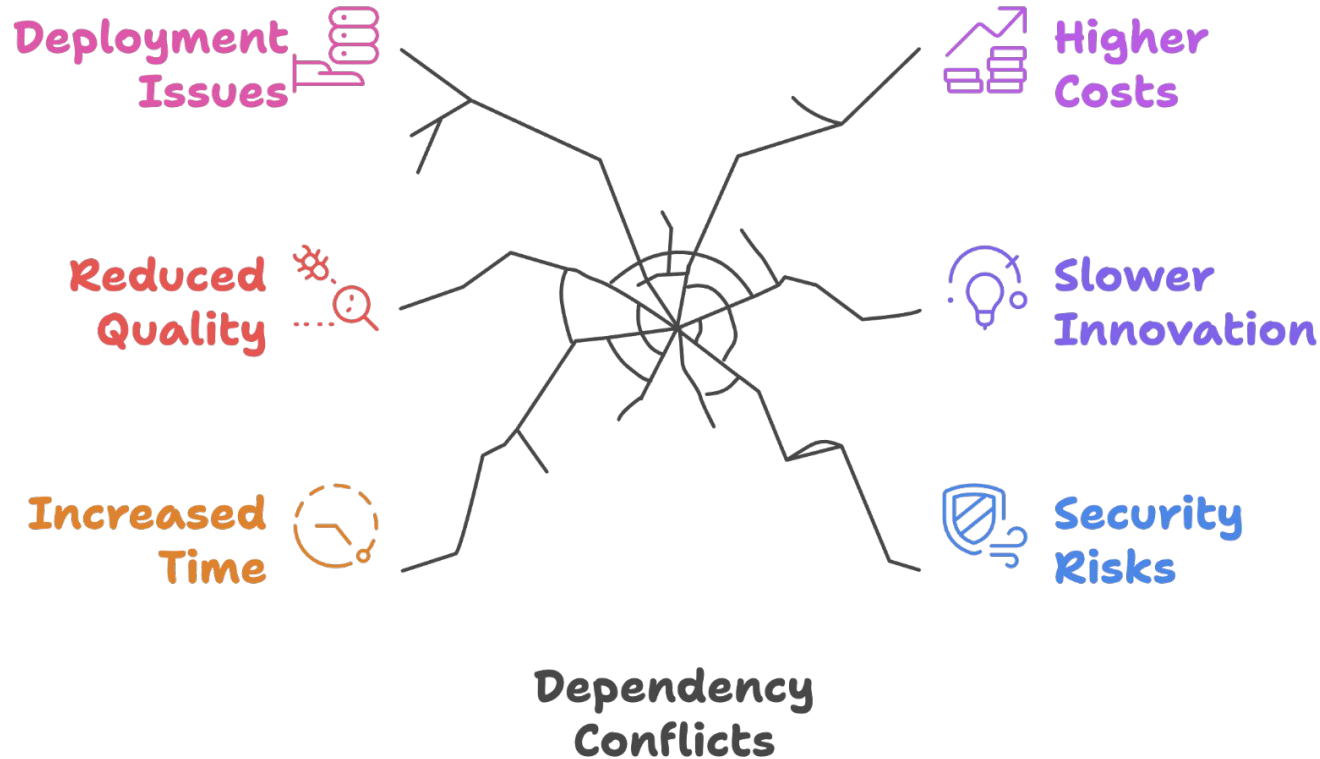


**IT'S 2025
WHY
TALK
ABOUT
THIS?**

Developers struggle to keep up with frequent updates.



Adopting new software



JVM and OSS libraries evolve faster



Spring Boot helps you to create Spring-powered, production-grade applications and services with absolute minimum fuss. It takes an opinionated view of the Spring platform so that new and existing users can quickly get to the bits they need.

Release	Released	OSS support	Commercial Support	Latest
3.3	3 months and 3 weeks ago (23 May 2024)	Ends in 8 months (23 May 2025)	Ends in 1 year and 11 months (23 Aug 2026)	3.3.3 (22 Aug 2024)
3.2	9 months and 4 weeks ago (23 Nov 2023)	Ends in 2 months and 1 week (23 Nov 2024)	Ends in 1 year and 5 months (23 Feb 2026)	3.2.9 (22 Aug 2024)
3.1	1 year and 4 months ago (18 May 2023)	Ended 4 months ago (18 May 2024)	Ends in 11 months (18 Aug 2025)	3.1.12 (23 May 2024)
3.0	1 year and 9 months ago (24 Nov 2022)	Ended 9 months ago (24 Nov 2023)	Ends in 5 months (24 Feb 2024)	
2.7	2 years and 4 months ago (19 May 2022)	Ended 9 months ago (24 Nov 2023)	Ends in 11 months (24 Aug 2024)	
2.6	2 years and 10 months ago (17 Nov 2021)	Ended 1 year and 9 months ago (24 Nov 2022)	Ended 6 months and 3 weeks ago (24 Feb 2023)	

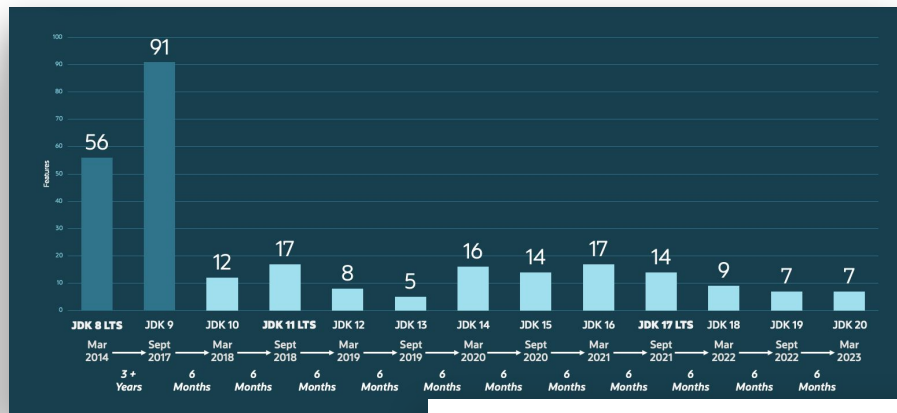


FIGURE 1.2. OPEN SOURCE NEW PROJECT GROWTH RATE OVER THE PAST 4 YEARS

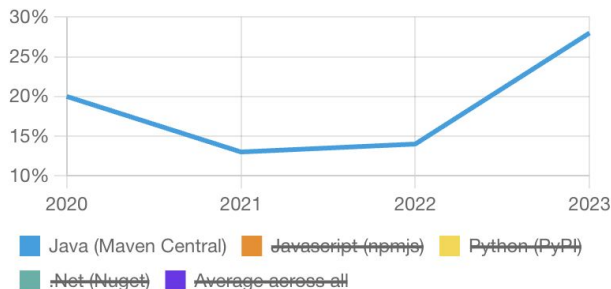
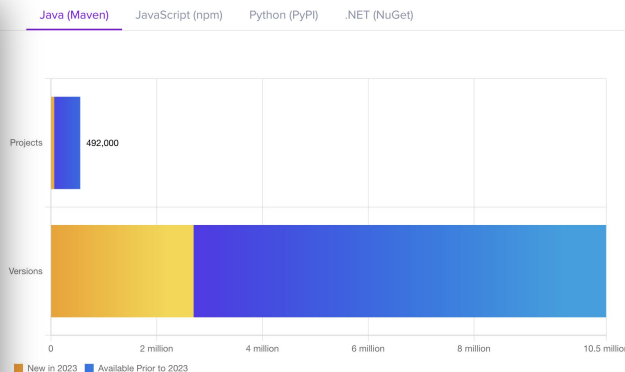


FIGURE 1.3. OPEN SOURCE PROJECTS AND VERSIONS GROWTH, 2023



TECHNICAL DEBT ACCUMULATES



**NETFLIX IS NOT
IMMUNE TO THIS,
LET'S TALK
ABOUT IT!**

NE



LIX

We have the paved path



Image from https://media.wired.com/photos/5926e050cefb457b079b6b6/master/w_2400%2Cc_limit/GettyImages-166135659.jpg

JVM Backend landscape



Gradle



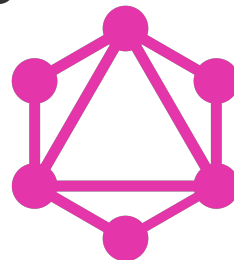
ELASTICSEARCH



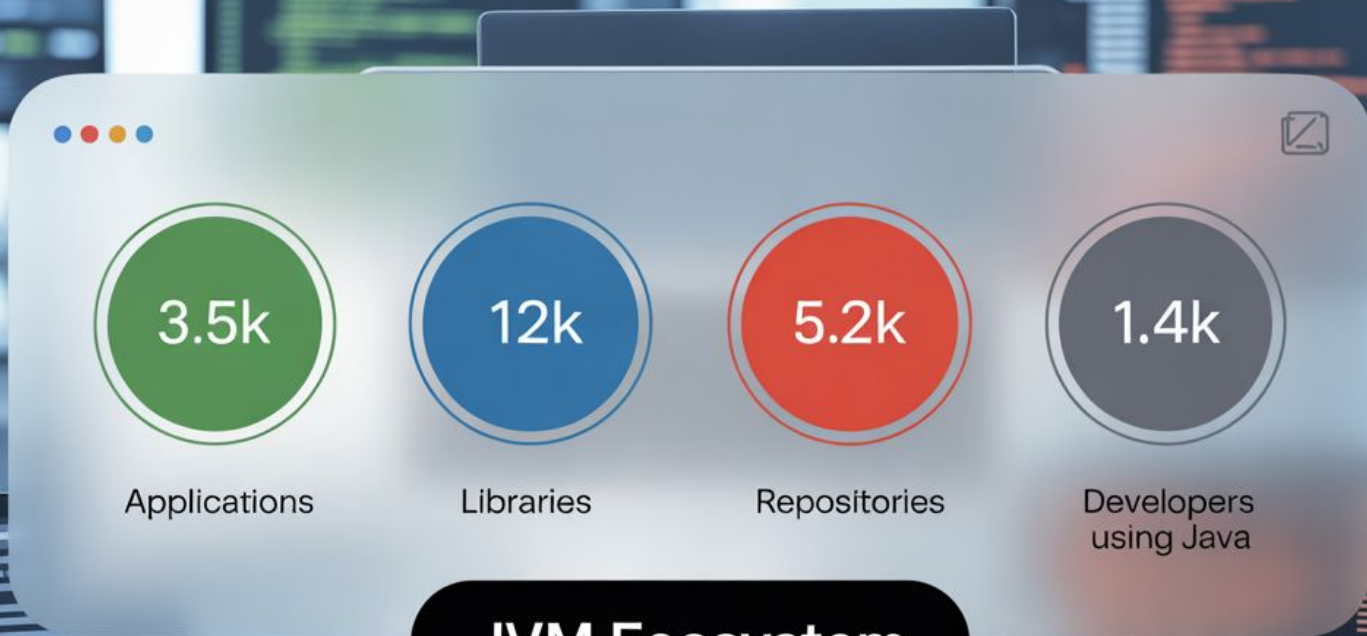
EV cache



DGS

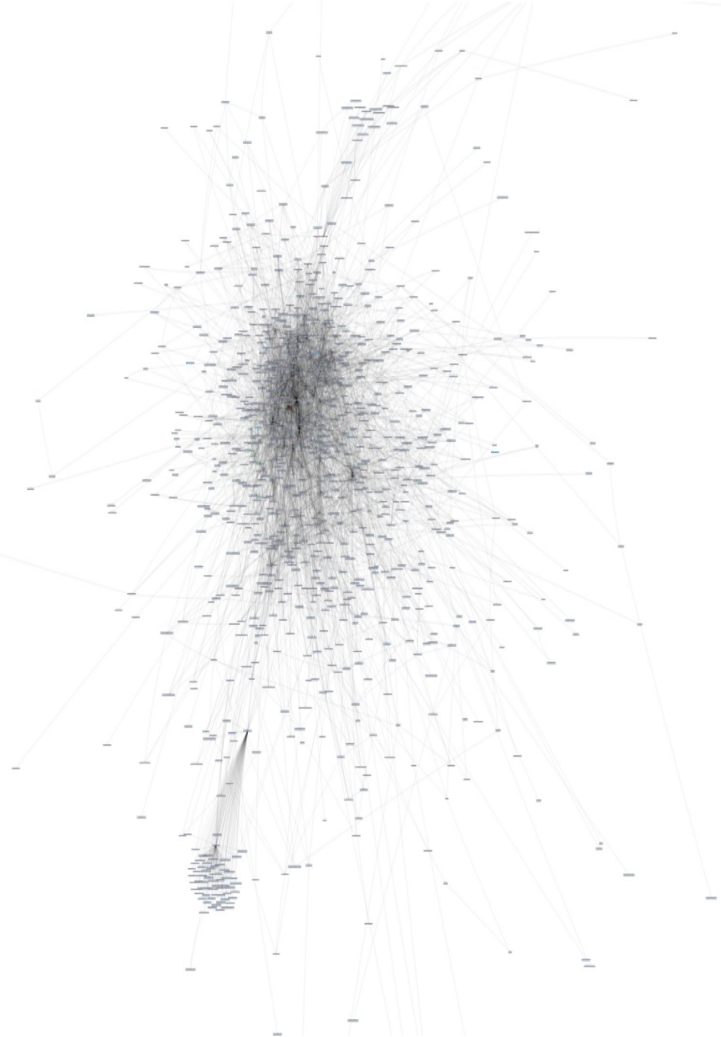


NETFLIX



JVM Ecosystem

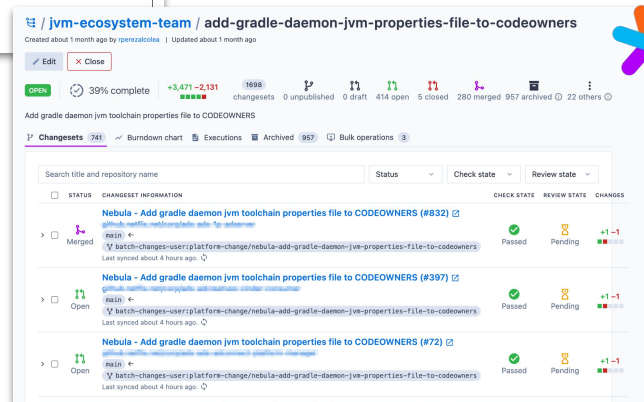
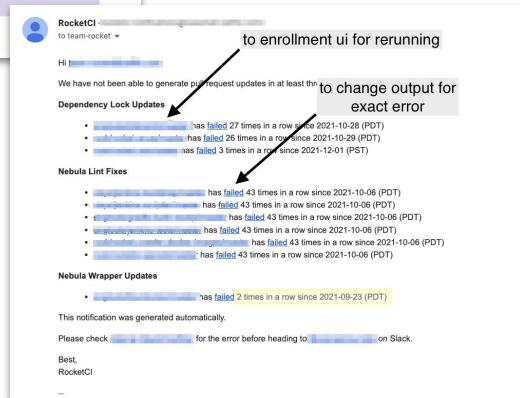
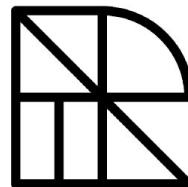
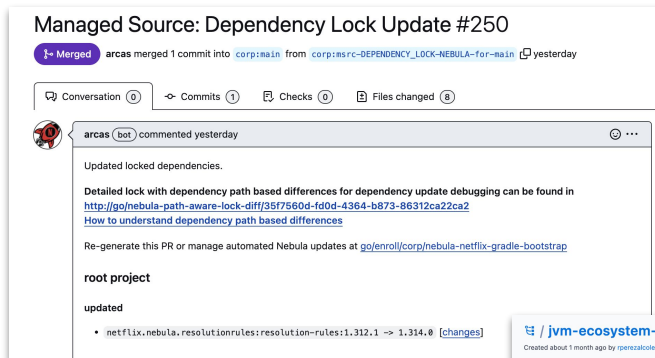
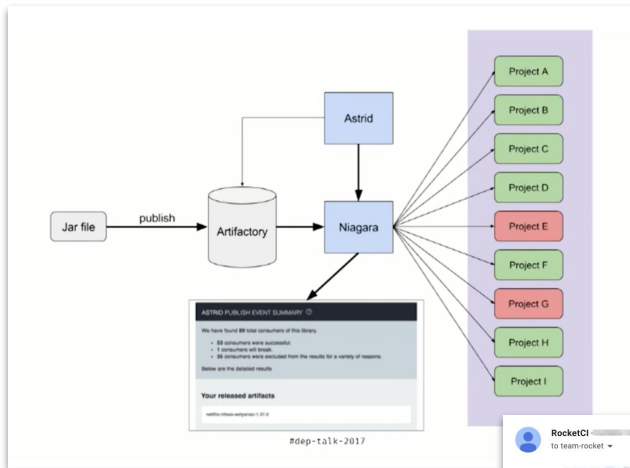
Eventually consistent distributed monolith



We had to invest on delivering change at scale



Examples of Investments



**Turns out
delivering code
changes is hard!**





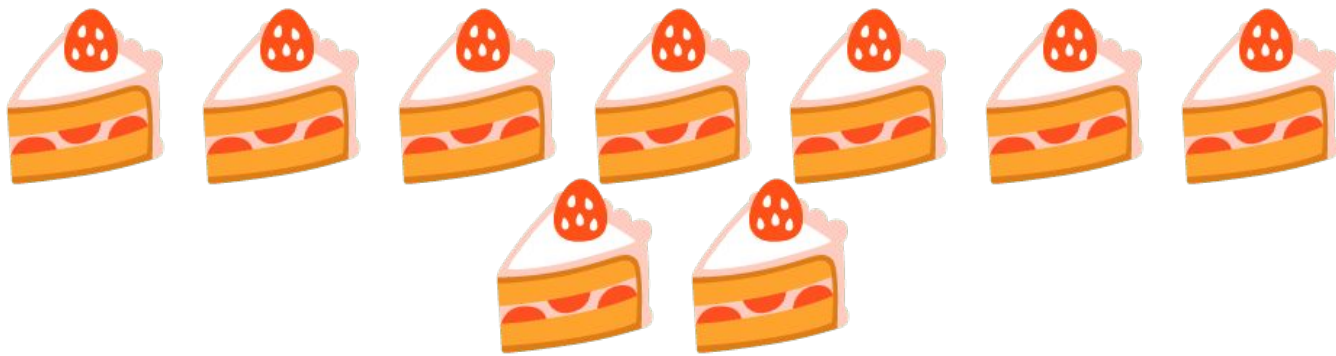
**Let's look at
a real world
example**



A very small code change

```
1 java {  
2     toolchain {  
3         languageVersion = JavaLanguageVersion.of(17)  
4     }  
5 }
```

to upgrade the JDK for application runtime



**Very easy...
right?**



**Unfortunately,
IT IS NOT 🙄...
let's look at
some learnings**



Existing verification might not be sufficient

 Pinned by incident

 **incident** APP 2:27 PM

JDK17 upgrade preventing batch send for [REDACTED]

Problem: The upgrade to JDK17 is preventing the batch send process for [REDACTED].

Impact: Unknown

Causes: Unknown

Steps to resolve: We pinned to a previous version of [REDACTED] in order to proceed with the revert, then re-ran the segment.

 Page a team

 Set fields

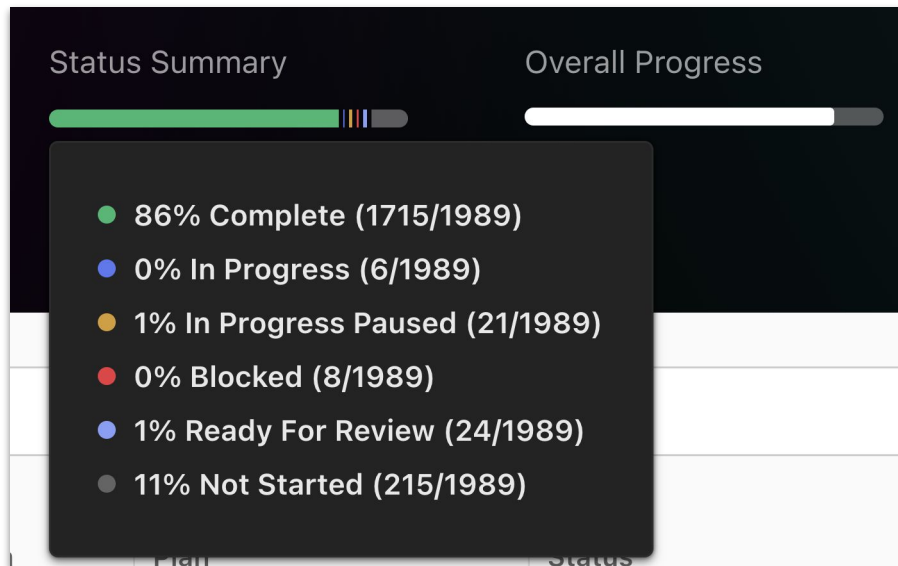
 View all commands

 Overview

And human-watched or human-implemented migrations take time



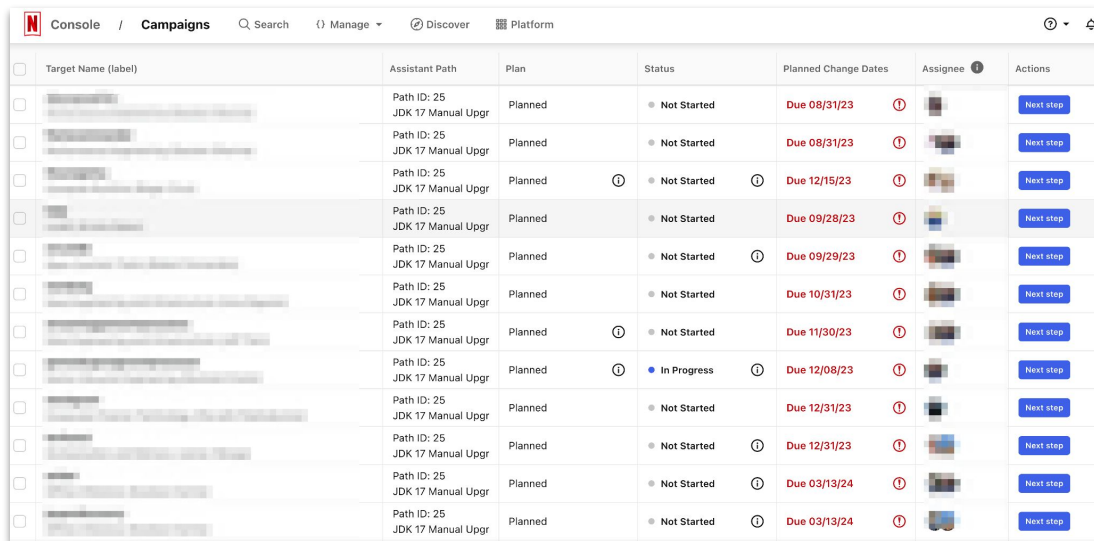
The campaign targets (size)



Thousands of targeted applications and each one of them has 1 or more running instances

Requires owner engagement

1. Teams have their own priorities
2. Providing automation is great but they still wanted to have control of the change
3. Requires scaling for providing great support and customer experience



The screenshot displays a web interface for managing campaigns. At the top, there is a navigation bar with a logo, 'Console / Campaigns', a search bar, and several icons. Below this is a table with columns: Target Name (label), Assistant Path, Plan, Status, Planned Change Dates, Assignee, and Actions. The table contains 12 rows of data, each representing a planned change. Most changes are 'Not Started', with one being 'In Progress'. Each row includes a 'Next step' button in the Actions column.

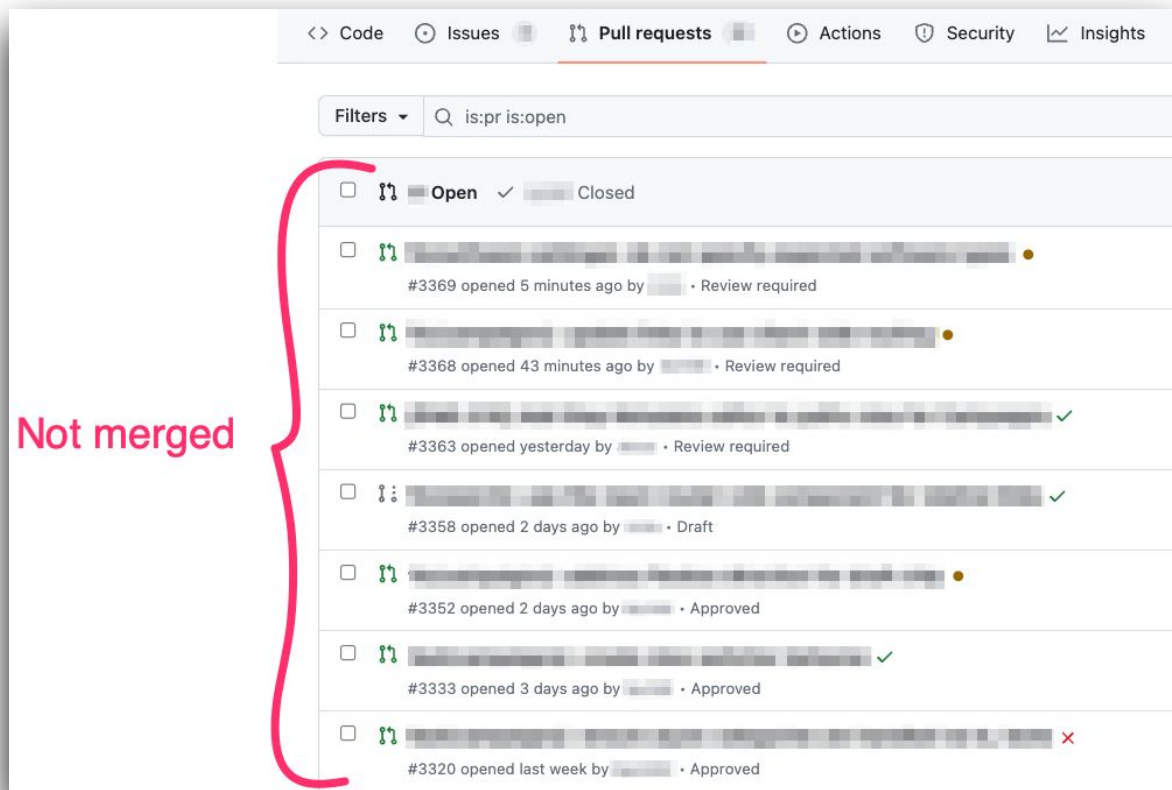
Target Name (label)	Assistant Path	Plan	Status	Planned Change Dates	Assignee	Actions
[Redacted]	Path ID: 25 JDK 17 Manual Upgr	Planned	Not Started	Due 08/31/23	[Redacted]	Next step
[Redacted]	Path ID: 25 JDK 17 Manual Upgr	Planned	Not Started	Due 08/31/23	[Redacted]	Next step
[Redacted]	Path ID: 25 JDK 17 Manual Upgr	Planned	Not Started	Due 12/15/23	[Redacted]	Next step
[Redacted]	Path ID: 25 JDK 17 Manual Upgr	Planned	Not Started	Due 09/28/23	[Redacted]	Next step
[Redacted]	Path ID: 25 JDK 17 Manual Upgr	Planned	Not Started	Due 09/29/23	[Redacted]	Next step
[Redacted]	Path ID: 25 JDK 17 Manual Upgr	Planned	Not Started	Due 10/31/23	[Redacted]	Next step
[Redacted]	Path ID: 25 JDK 17 Manual Upgr	Planned	Not Started	Due 11/30/23	[Redacted]	Next step
[Redacted]	Path ID: 25 JDK 17 Manual Upgr	Planned	In Progress	Due 12/08/23	[Redacted]	Next step
[Redacted]	Path ID: 25 JDK 17 Manual Upgr	Planned	Not Started	Due 12/31/23	[Redacted]	Next step
[Redacted]	Path ID: 25 JDK 17 Manual Upgr	Planned	Not Started	Due 12/31/23	[Redacted]	Next step
[Redacted]	Path ID: 25 JDK 17 Manual Upgr	Planned	Not Started	Due 03/13/24	[Redacted]	Next step
[Redacted]	Path ID: 25 JDK 17 Manual Upgr	Planned	Not Started	Due 03/13/24	[Redacted]	Next step

**And this is just our
application JDK
campaign...**

**What about
more and
simultaneous
campaigns?** 🤔



Observations: engagement decreases



Observations: Deployments are not promoted

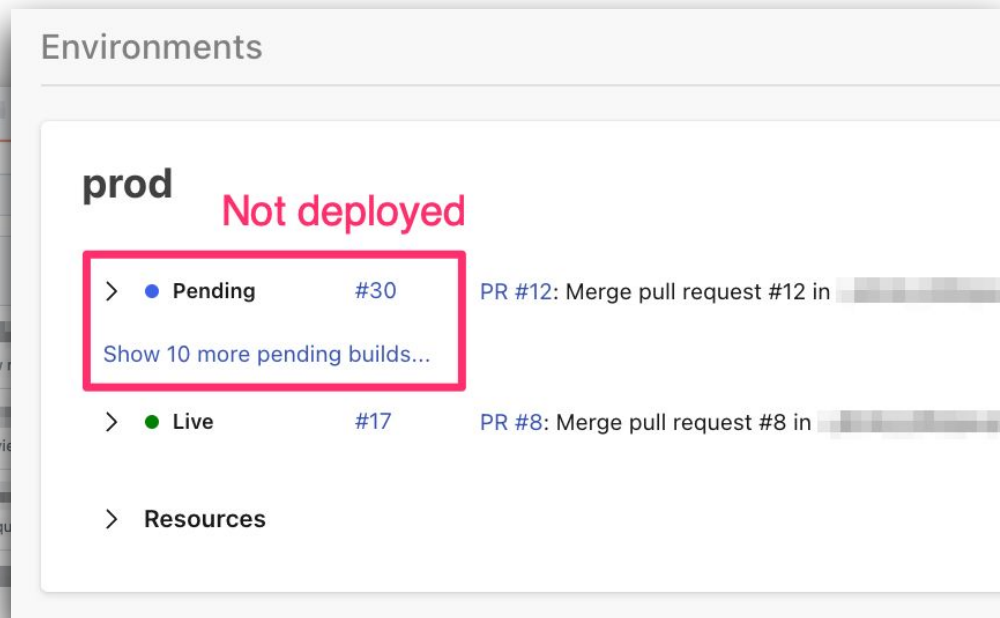
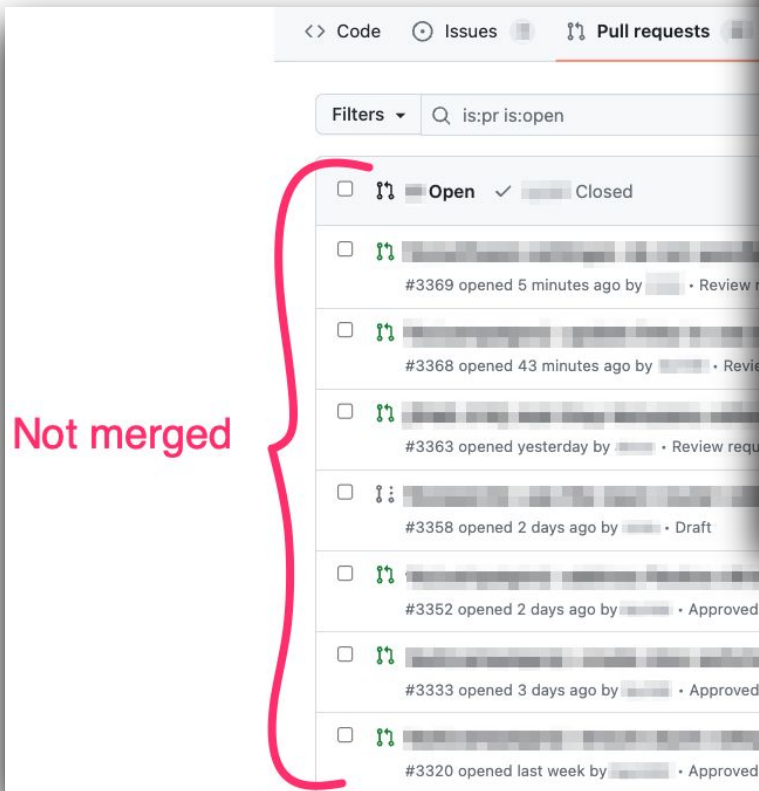
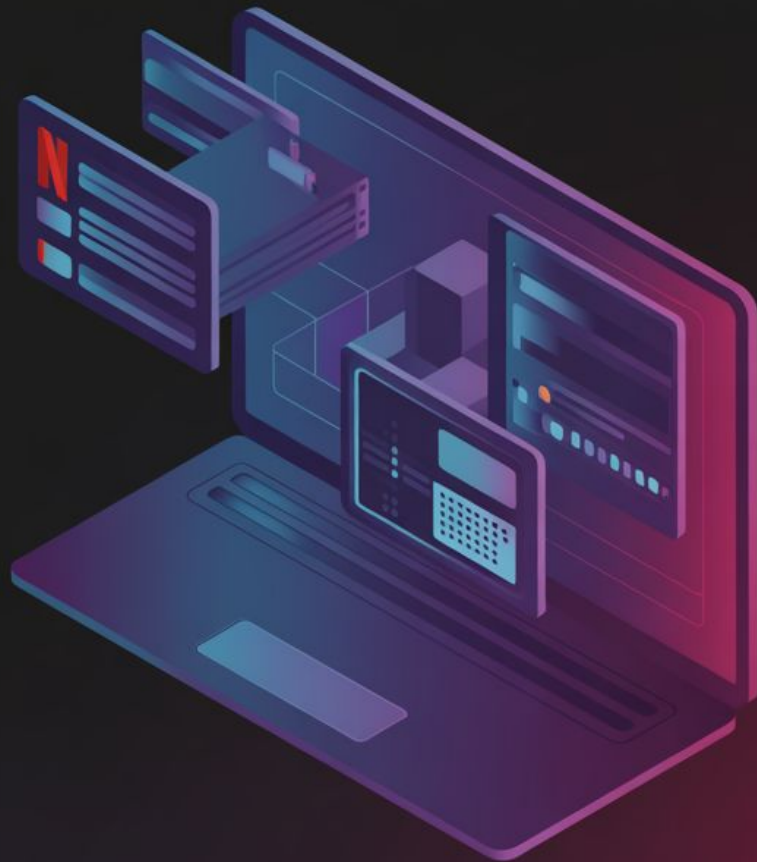


Image from screenshots within SCM tool
and within Deployment UI with Spinnaker

NETFLIX

How to improve Delivering Changes at Scale?



Foundations of Delivering Changes at Scale

Build Opinions



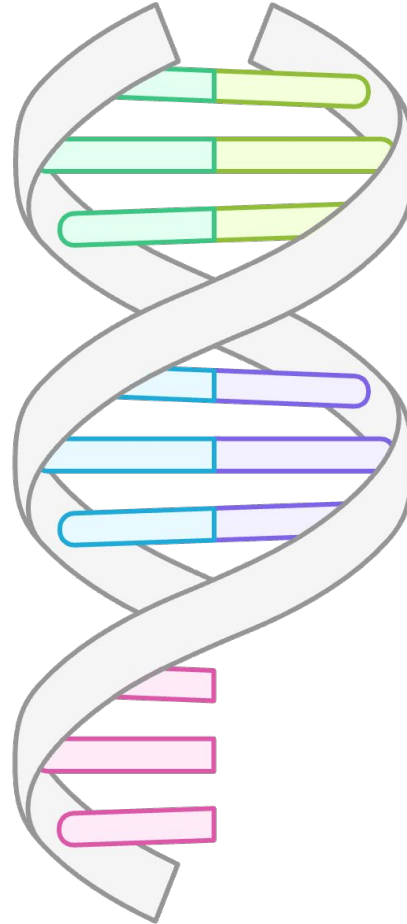
Artifact
Observability

Distributed
Refactoring



Continuous
Delivery

Testing and
Validation



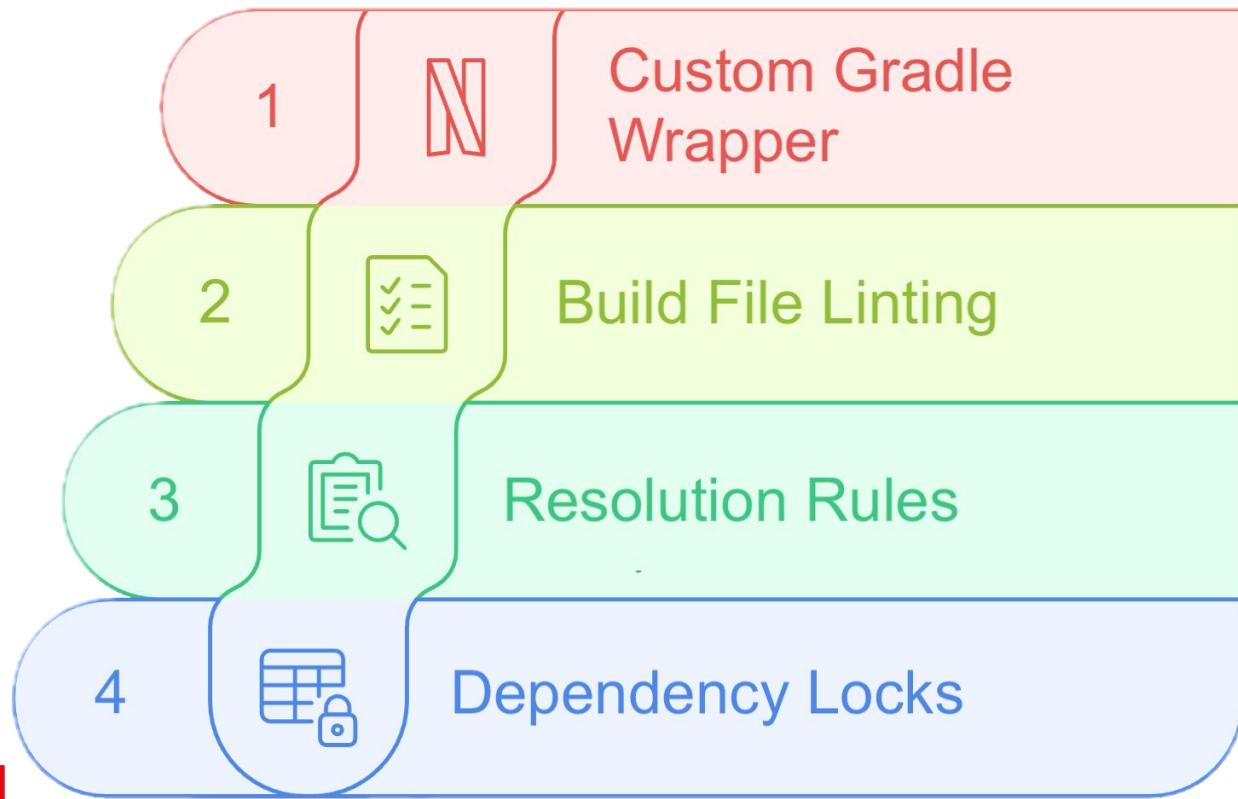
NETFLIX

Build Opinions[↗]

**PAVED
PATH**



Improving Dependency Management at Build Tooling Level



Gradle



nebula

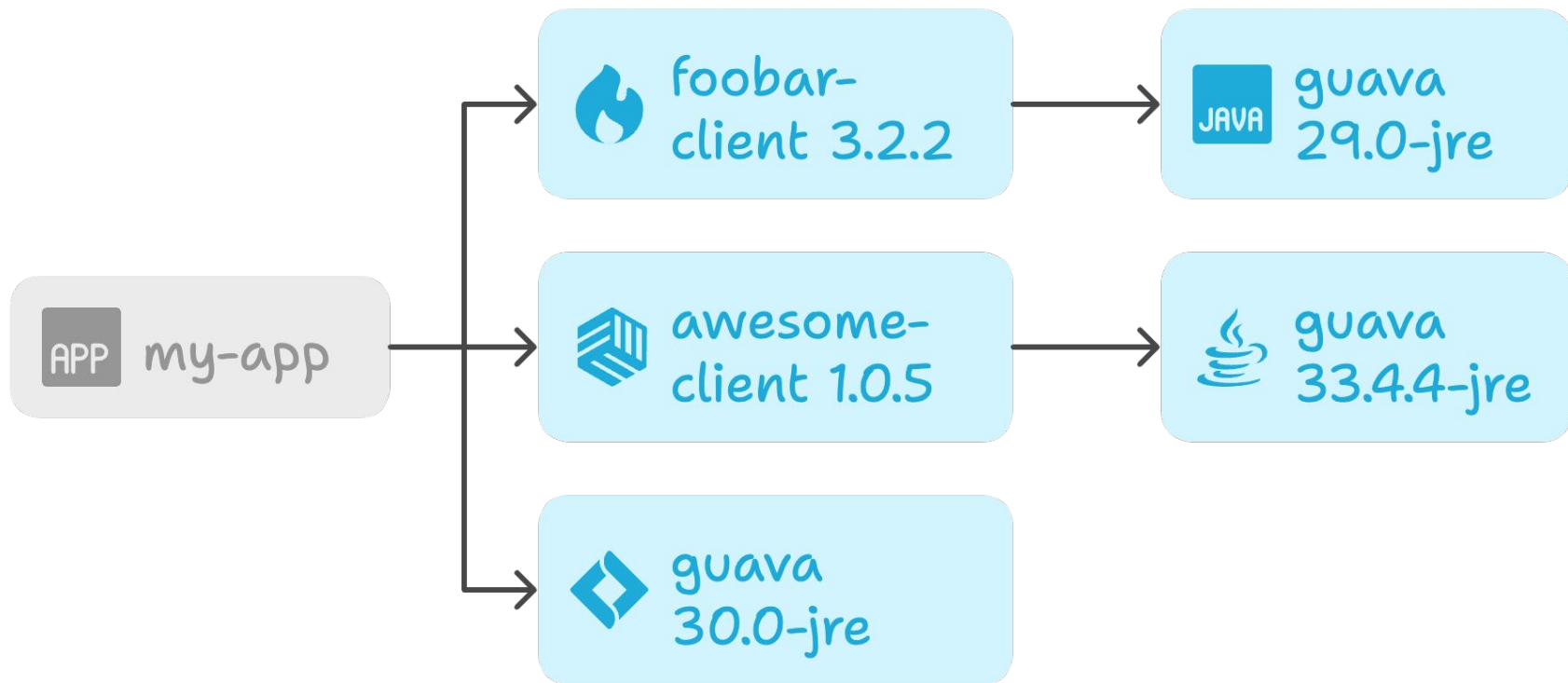
How engineers consume libraries



```
dependencies {  
  implementation 'com.netflix.internal:spring-boot-netflix:latest.release'  
  implementation 'com.netflix.teama:client-library:latest.release'  
  implementation 'com.netflix.internal:internal-library:latest.release'  
  implementation 'org.apache.logging.log4j:log4j-core:2.+'  
}
```

Consuming is easy right?

Software Package Dependency Diagram



NETFLIX

We Introduced
Nebula
Dependency
Locking in
Builds

Image generated with <https://ideogram.ai/>



N

This is how it looks like

```
{
  "annotationProcessor": {
    "org.projectlombok:lombok": {
      "locked": "1.18.38"
    }
  },
  "compileClasspath": {
    "org.projectlombok:lombok": {
      "locked": "1.18.38"
    }
  },
  "testAnnotationProcessor": {
    "org.projectlombok:lombok": {
      "locked": "1.18.38"
    }
  },
  "testCompileClasspath": {
    "org.projectlombok:lombok": {
      "locked": "1.18.38"
    }
  }
}
```


A modern desk setup with a computer monitor, keyboard, mouse, lamp, and various decorative items. The monitor displays the text "But what about Transitive Dependencies?". The desk is dark wood, and the background is a dark blue wall. A framed picture of a building is on the wall to the left. A potted plant is on the desk to the left of the monitor. A stack of books and a small notepad are on the desk to the left of the monitor. A desk lamp is on the right side of the desk. A small wooden shelf holds a stack of books and a small screen displaying the word "NETFLIX".

But what about
**Transitive
Dependencies?**

NETFLIX

We could have polluted builds



```
configurations.all {  
    resolutionStrategy {  
        force 'com.google.guava:guava:33.4.8-jre'  
    }  
}
```

Or distribute this through plugins 🤔

NETFLIX

INSTEAD, WE BUILT RESOLUTION RULES PLUGIN

ALIGN

SUBSTITUTE

DENY

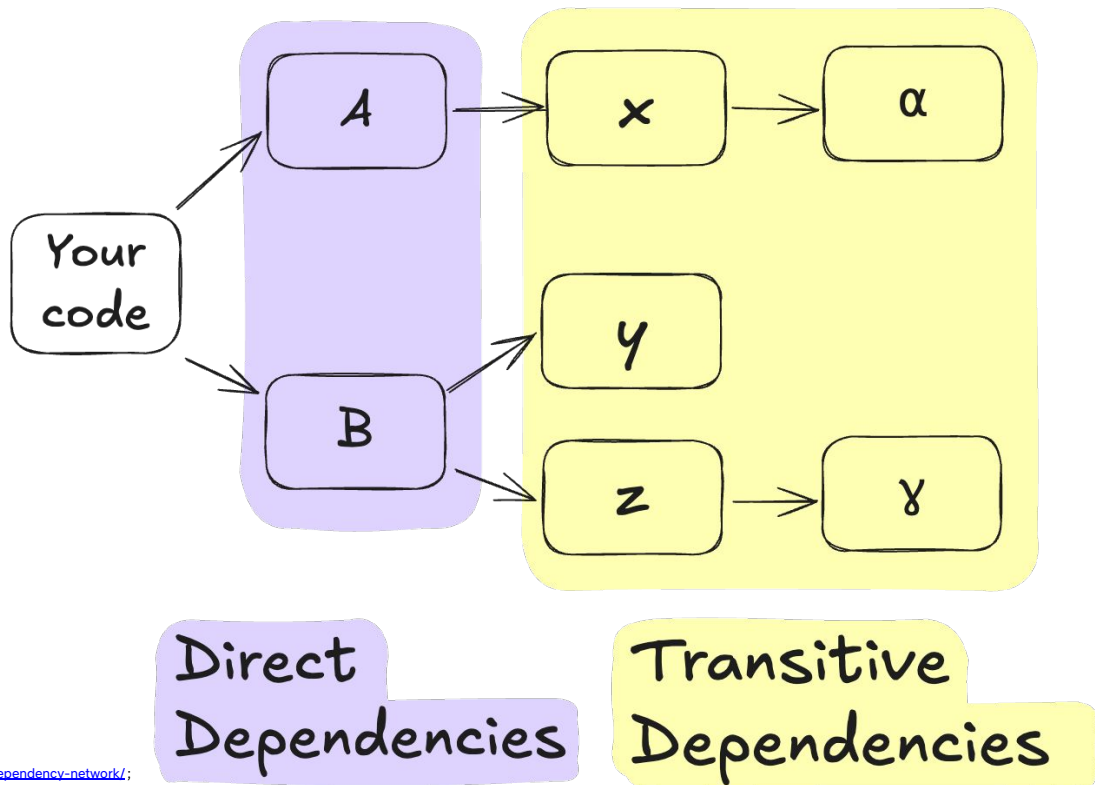
REJECT

REPLACE

Resolution Rules



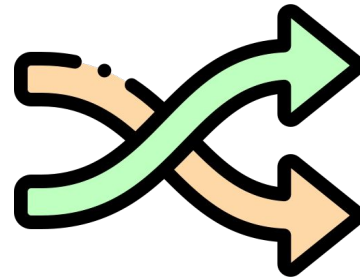
Modify direct and transitive dependencies at resolution time





Resolution Rules

Substitution



Plugin available at <https://github.com/nebula-plugins/gradle-resolution-rules-plugin>

Resolution Rules: Substitution

Substitute at resolution time

Index of com.netflix:a

1.0.0

1.0.1

1.0.2

1.1.0

1.2.0

1.3.0

1.4.0

1.5.0

1.6.0

1.6.1

com.netflix:a:1.2.0

first version with an issue

com.netflix:a:1.5.0

first version with a fix

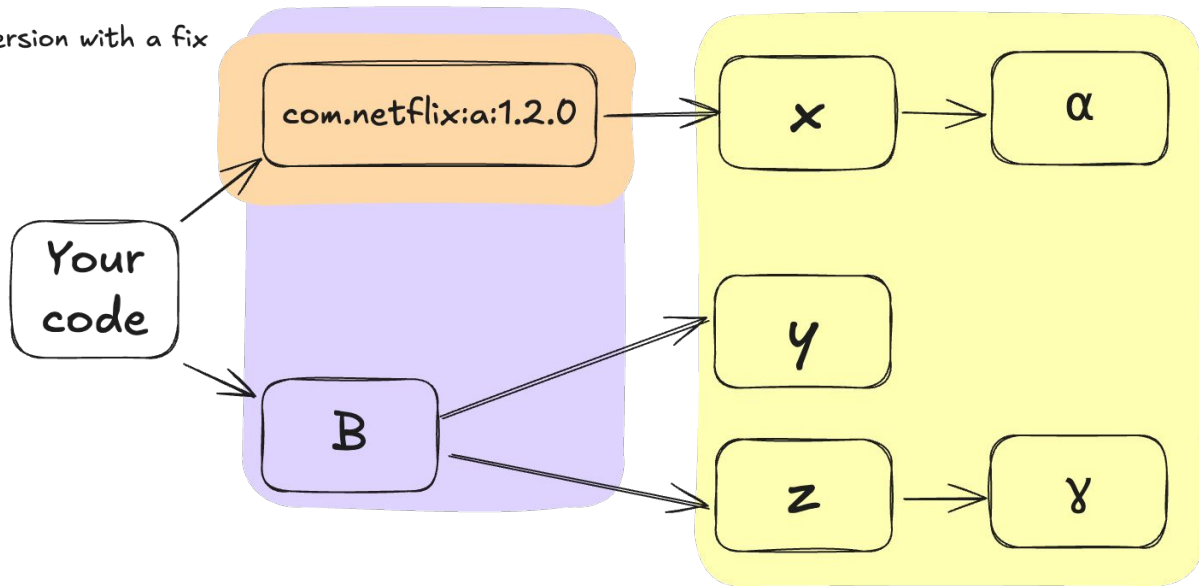
Resolution Rules: Substitution

com.netflix:a:1.2.0

first version with an issue

com.netflix:a:1.5.0

first version with a fix



Direct
Dependencies

Transitive
Dependencies

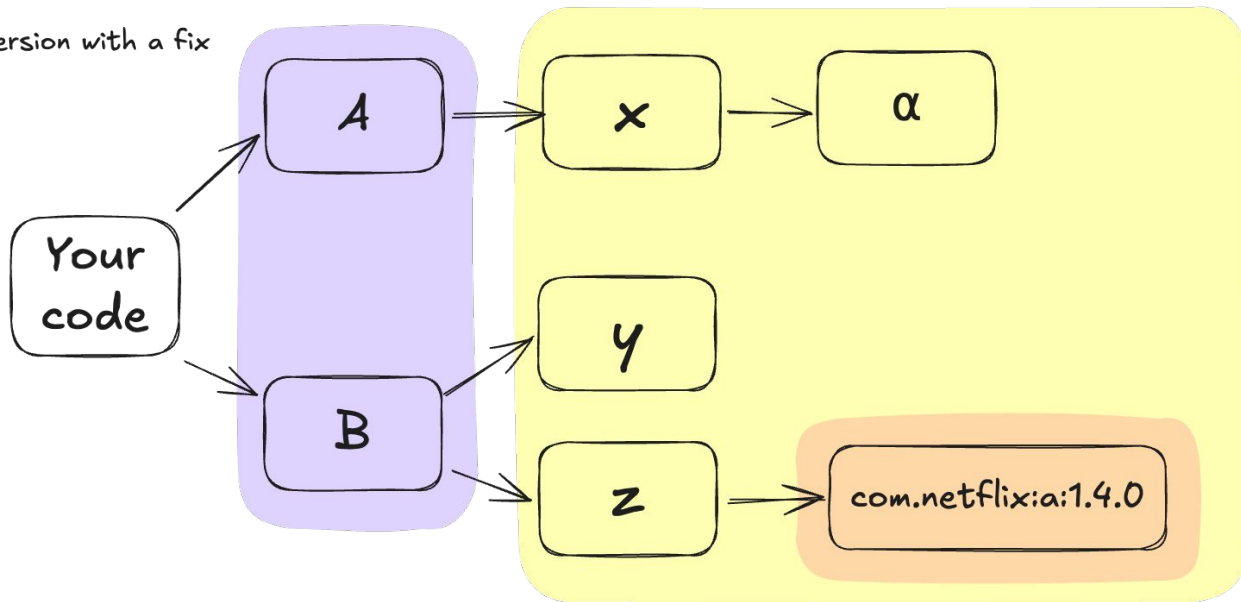
Resolution Rules: Substitution

com.netflix:a:1.2.0

first version with an issue

com.netflix:a:1.5.0

first version with a fix



Direct
Dependencies

Transitive
Dependencies

Resolution Rules: Substitution

Substitute at resolution time

Index of com.netflix:a

1.0.0

1.0.1

1.0.2

1.1.0

1.2.0

1.3.0

1.4.0

1.5.0

1.6.0

1.6.1

com.netflix:a:1.2.0

first version with an issue

com.netflix:a:1.5.0

first version with a fix

Version range with an issue:

com.netflix:a:[1.2.0,1.5.0)

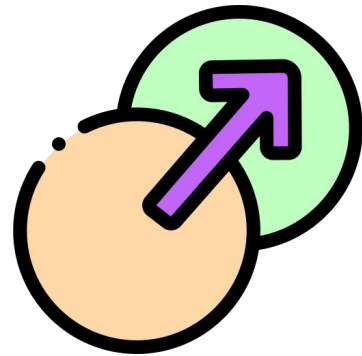
Version to substitute-towards:

com.netflix:a:1.5.0



Resolution Rules

Replacement



Plugin available at <https://github.com/nebula-plugins/gradle-resolution-rules-plugin>

Resolution Rules: Replacement

Replace a dependency at resolution time when both are provided

`io.grpc:grpc-api`

replace from this

`com.netflix.io.grpc:grpc-api`

to this

Both should not be in the classpath

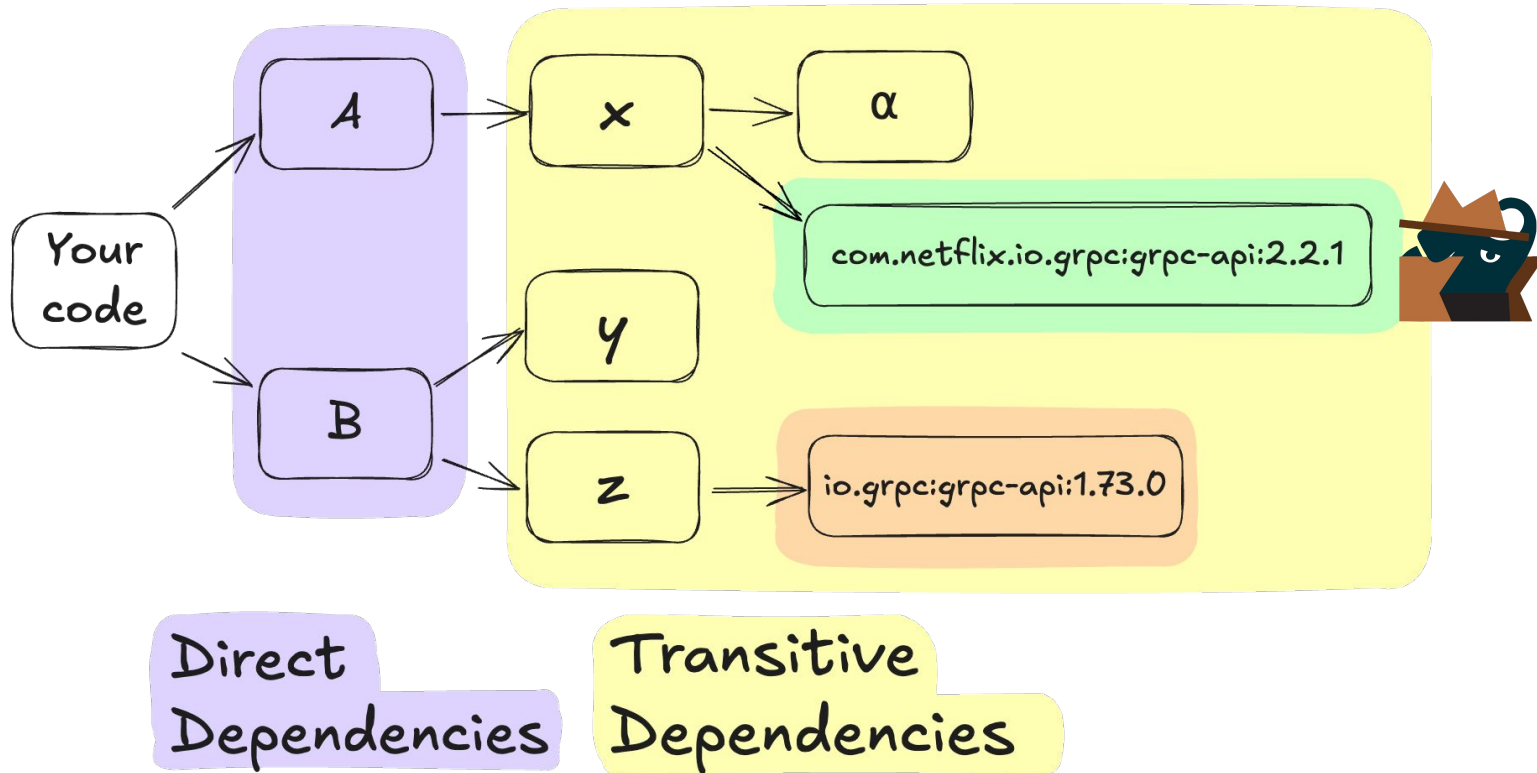
They provide the same classes

We can use the internal flavor with shaded dependencies



Resolution Rules: Replacement

Replace a dependency at resolution time when both are provided



Resolution Rules: Replacement

Replace a dependency at resolution time when both are provided

`io.grpc:grpc-api`

replace from this

`com.netflix.io.grpc:grpc-api`

to this

Dependency moving away from:

`io.grpc:grpc-api`

Dependency to replace-towards:

`com.netflix.io.grpc:grpc-api` 



Resolution Rules

Alignment



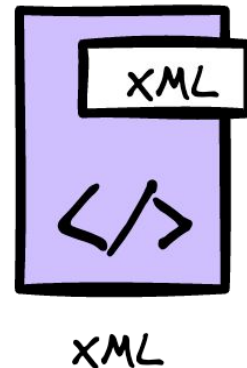
Plugin available at <https://github.com/nebula-plugins/gradle-resolution-rules-plugin>

Resolution Rules: Alignment



Align dependency versions at resolution time

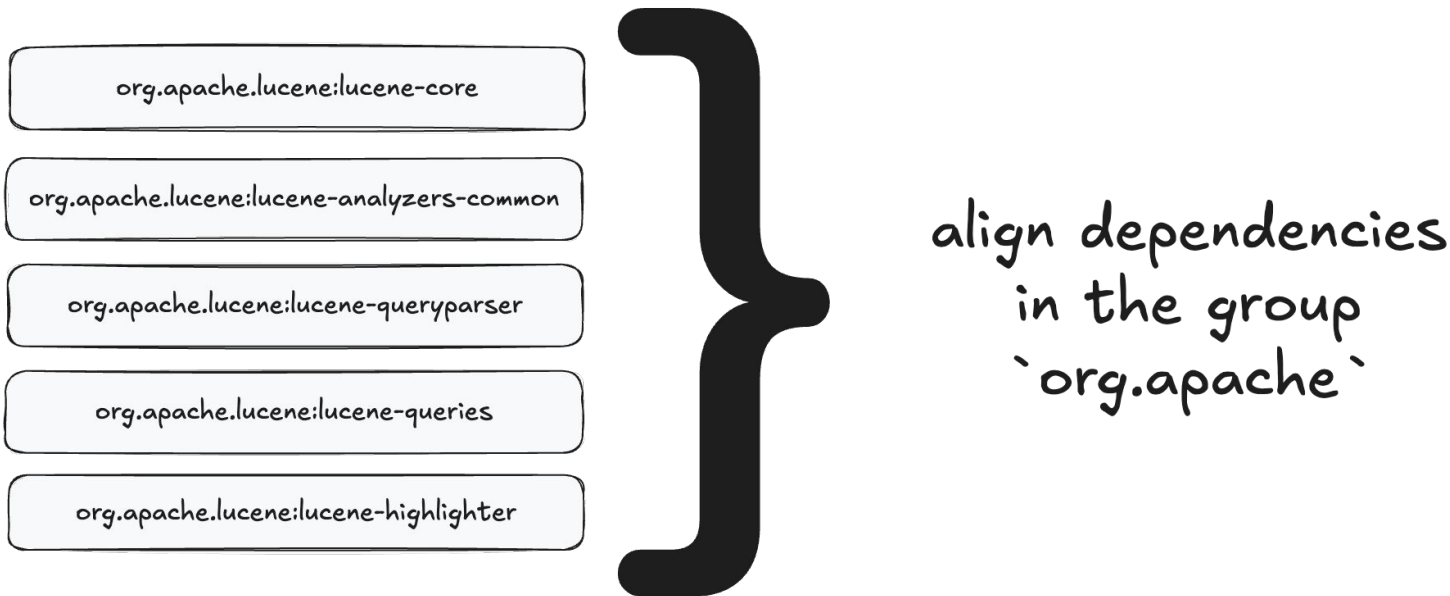
Similar to a Bill of Materials (BOM)
to manage and synchronize dependency
versions



Resolution Rules: Alignment



Align dependency versions at resolution time





Resolution Rules

Mechanisms



Plugin available at <https://github.com/nebula-plugins/gradle-resolution-rules-plugin>

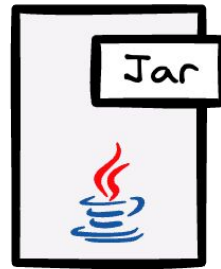
Resolution Rules: Mechanisms

{JSON}

substitute-com-netflix-a.json
substitute-.....json
substitute-.....json
substitute-.....json
replace-.....json
replace-.....json
align-.....json
align-.....json



resolution-rules.jar



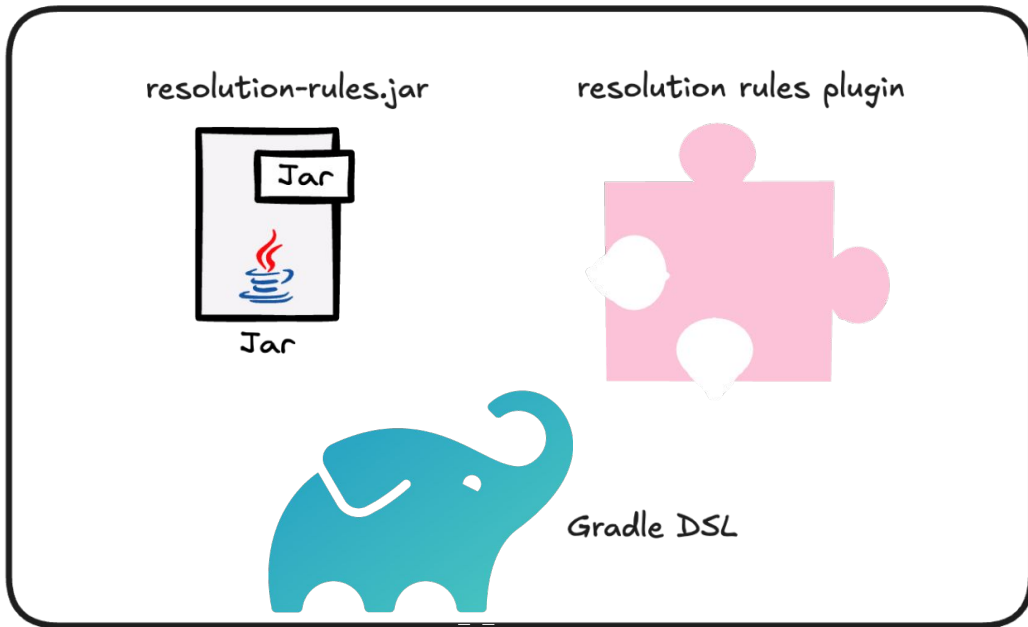
Jar

Resolution Rules: Mechanisms



Netflix Nebula

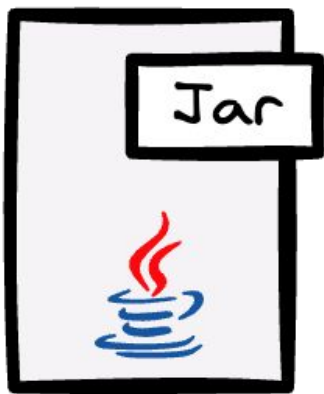
includes



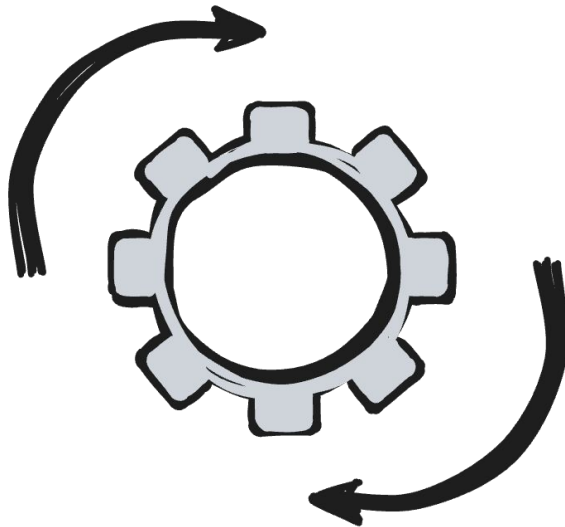
Resolution Rules: Mechanisms

When project dependencies are updated, we also update the versioned resolution-rules jar and apply its opinions

resolution-rules.jar



Jar



And we do all of this through Nebula!

A custom Gradle distribution

```
distributionBase=GRADLE_USER_HOME  
distributionPath=wrapper/dists  
distributionUrl=https\://artifactory/nebula-gradle/9.16.6/nebula-gradle-9.16.6-bin.zip  
networkTimeout=10000  
validateDistributionUrl=true  
zipStoreBase=GRADLE_USER_HOME  
zipStorePath=wrapper/dists
```





Resolution Rules and Nebula Distribution



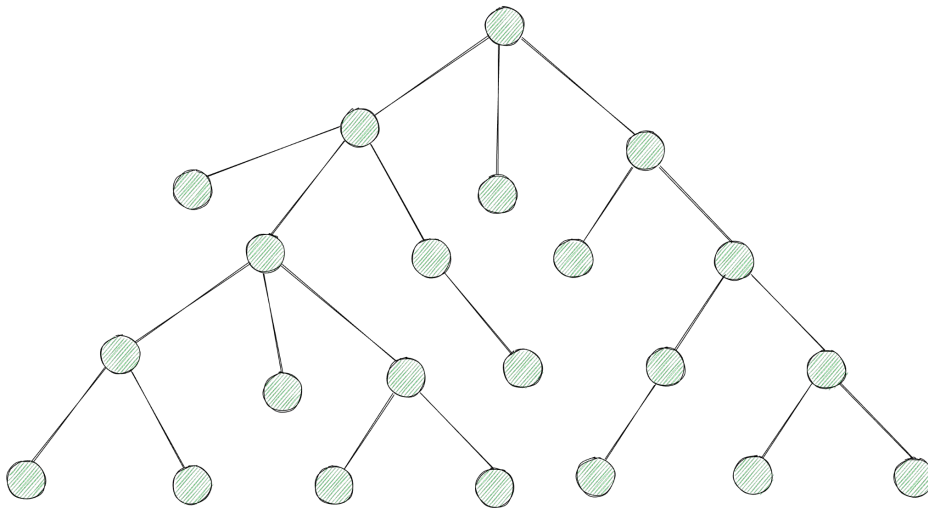
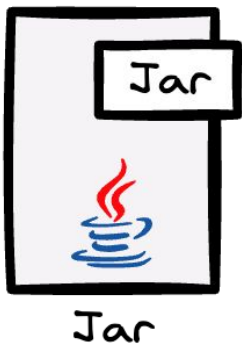
Validations

Nebula Distribution: Validations

How do we know this version of resolution rules is valid?

We run an acceptance test!

resolution-rules.jar



N

If consuming libraries
is hard, publishing
should be easier, right?

NOT REALLY!



N

**If I want to release
my library, I
want to answer:
Who is using it?
How is being used?
Who will I break?**

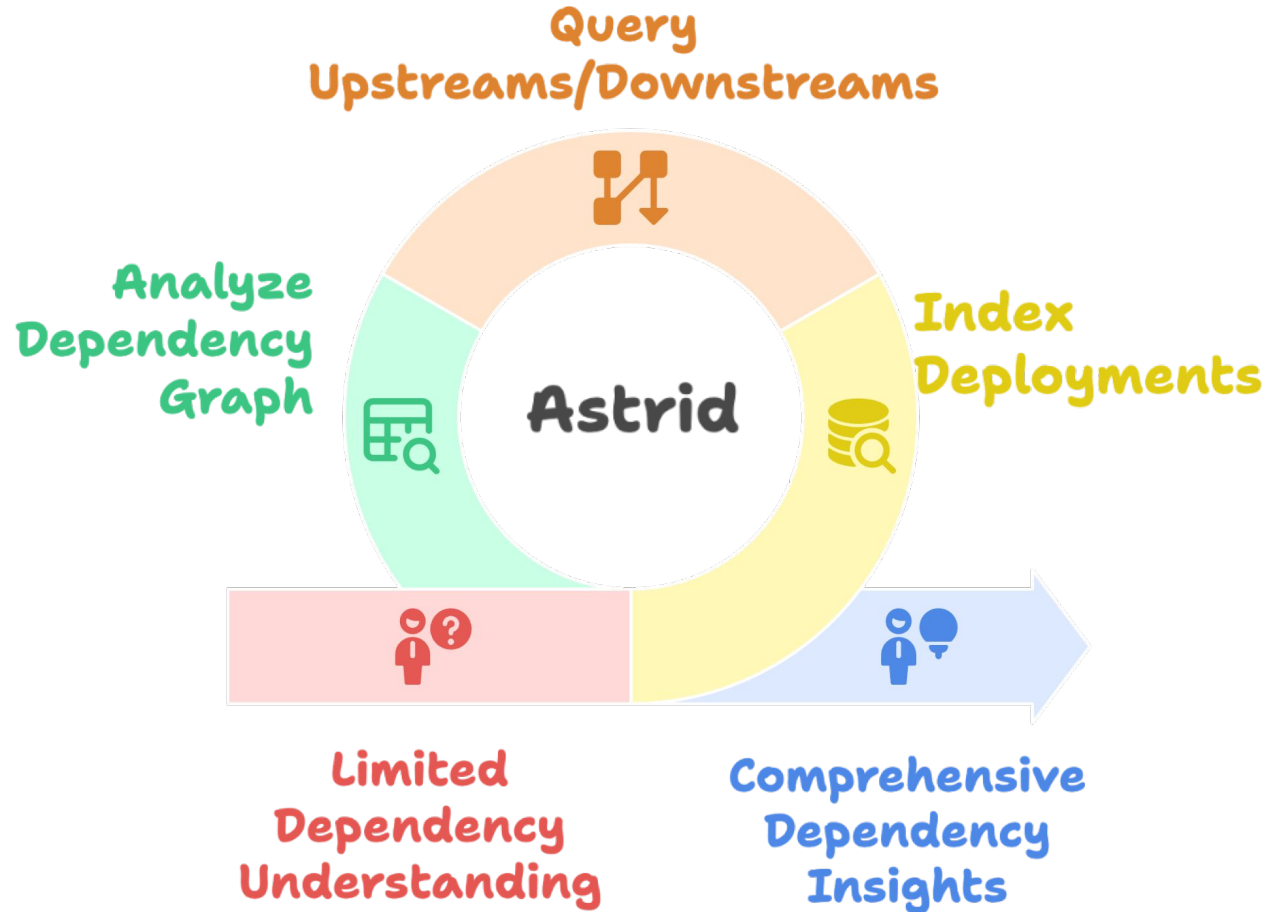




Artifact Observability

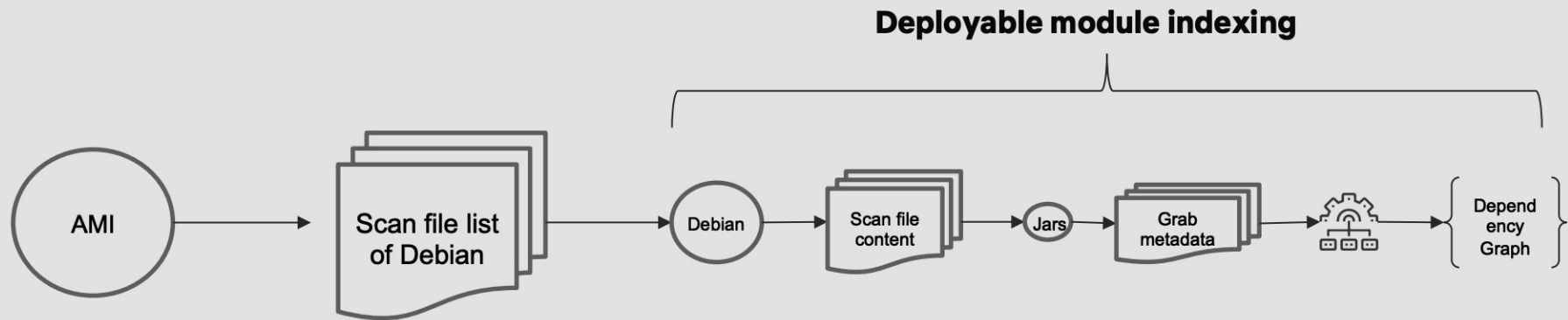


Achieving Artifact Observability with Astrid



How does it work?

Indexing of AMI (Amazon Machine Image) manifests



Understanding usage of a library

Artifact Management: com.google.guava/guava

Adoption Summary for 33.4.8-jre

☒ Include Transitive Usage

Deprecate <=33.4.8-jre

Block usage of 33.4.8-jre

Summary

Includes usage via transitive dependencies. [Show only direct usage](#)

USAGE FOR 33.4.8-JRE

All Consumer Server Groups(3248) 6.55% Adoption

6.5% adoption

Usage for exact version

100

93.5% adoption

USAGE FOR <33.4.8-JRE

All Consumer Server Groups(46321) 93.45% Adoption

[Show Complete List](#)

Usage for lower versions

100

1000

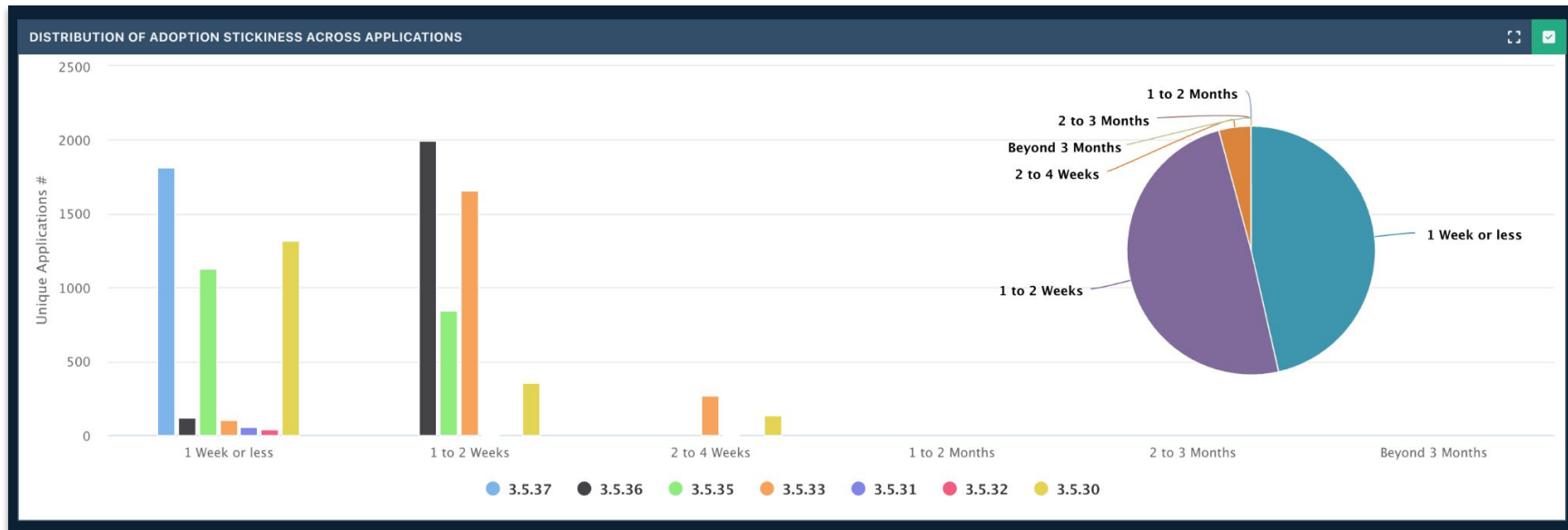
10000

Understanding adoption delay



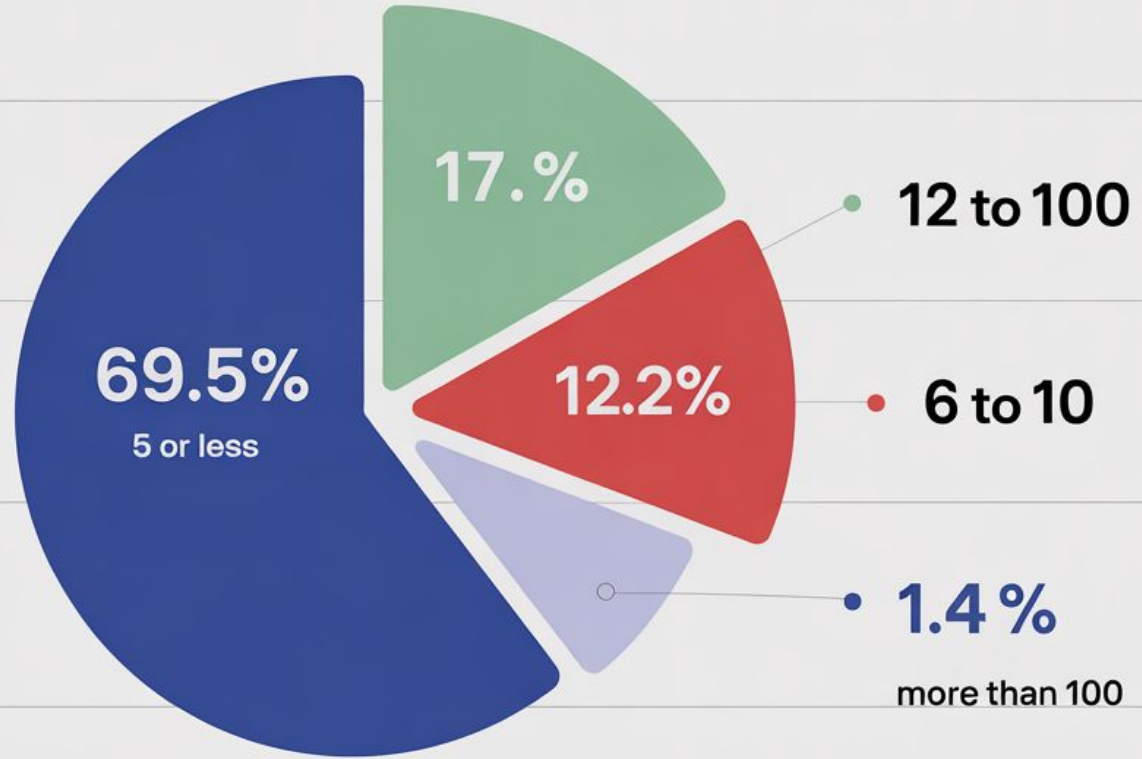
Adoption Delay is defined as the time period between when a module version was released and when an application started using the version

Understanding adoption stickiness



Adoption Stickiness is defined as the time period between when a module version was released and the last moment that an application was still using the version

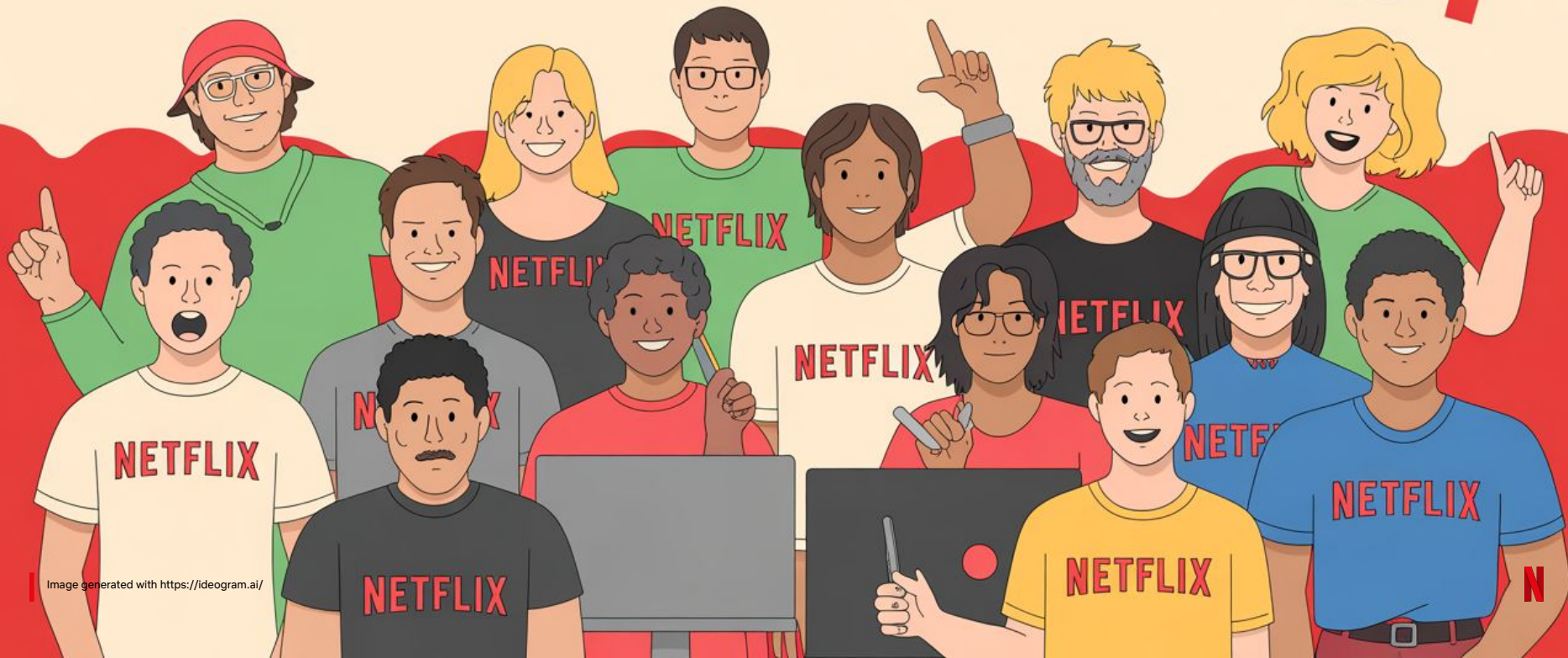
Distribution of modules by active versions



N

Do developers address
all migrations manually?

No!





Distributed Refactoring

Delivering changes

Nightly Updates

Refreshes all dependencies across all repositories, ensuring consistency.

Selective Updates

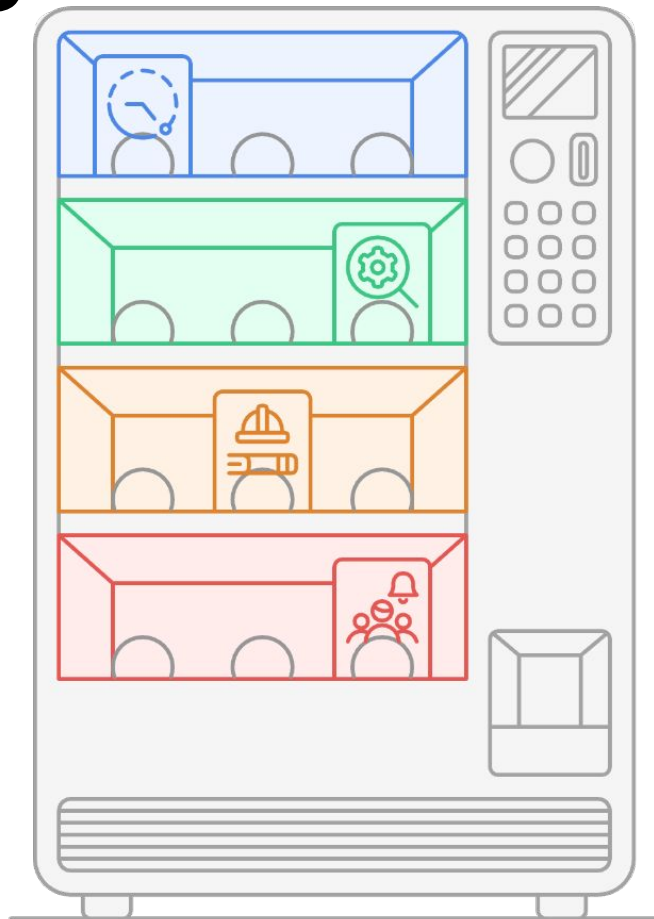
Allows targeted upgrades or downgrades of specific dependencies.

Migration Campaigns

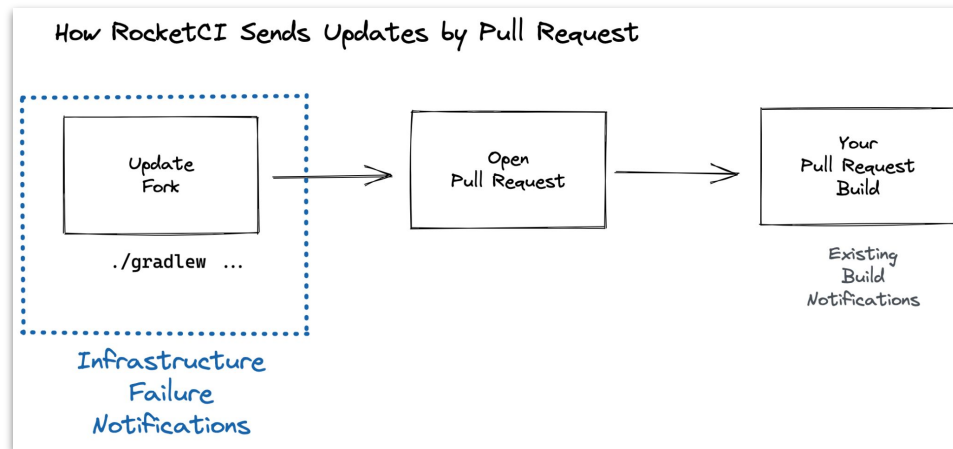
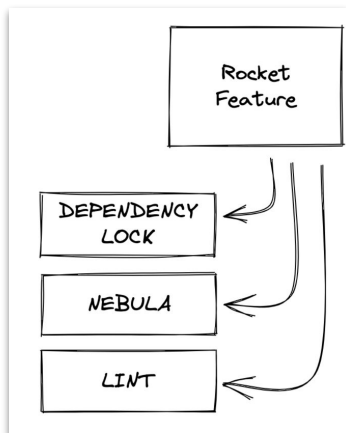
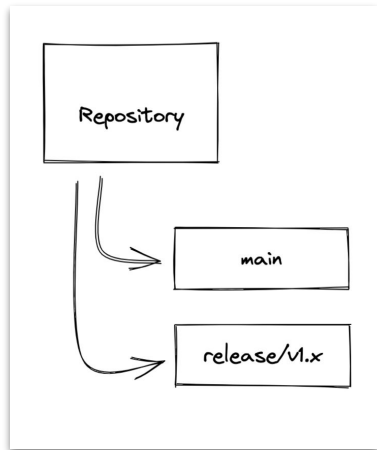
Addresses code changes required for complex updates.

Incident Campaigns

Responds to urgent issues like security vulnerabilities.



Rocket CI enrollments



Nightly dependency updates



Managed Source: Dependency Lock Update #250

Merged arcas merged 1 commit into corp:main from corp:msrc-DEPENDENCY_LOCK-NEBULA-for-main yesterday

Conversation (0) Commits (1) Checks (0) Files changed (8)



arcas (bot) commented yesterday

Updated locked dependencies.

Detailed lock with dependency path based differences for dependency update debugging
<http://go/nebula-path-aware-lock-diff/35f7560d-fd0d-4364-b873-86312ca22ca2>
[How to understand dependency path based differences](#)

Re-generate this PR or manage automated Nebula updates at go/enroll/corp/nebula-netflix-gr

root project

updated

- netflix.nebula.resolutionrules:resolution-rules:1.312.1 -> 1.314.0 [\[changes\]](#)



RocketCI

to team-rocket

to enrollment ui for rerunning

Hi

We have not been able to generate pull request updates in at least three

to change output for exact error

Dependency Lock Updates

- has failed 27 times in a row since 2021-10-28 (PDT)
- has failed 26 times in a row since 2021-10-29 (PDT)
- has failed 3 times in a row since 2021-12-01 (PST)

Nebula Lint Fixes

- has failed 43 times in a row since 2021-10-06 (PDT)
- has failed 43 times in a row since 2021-10-06 (PDT)
- has failed 43 times in a row since 2021-10-06 (PDT)
- has failed 43 times in a row since 2021-10-06 (PDT)
- has failed 43 times in a row since 2021-10-06 (PDT)
- has failed 43 times in a row since 2021-10-06 (PDT)

Nebula Wrapper Updates

- has failed 2 times in a row since 2021-09-23 (PDT)

This notification was generated automatically.

Please check for the error before heading to on Slack.

Best,
RocketCI

NETFLIX

Not everything should or
can be delivered overnight



WE RUN CAMPAIGNS



What is a campaign?

We track and coordinate the rollout across stakeholders from start to completion

N Console / Campaigns

Search Manage Discover Platform

← All campaigns

Increasing Confidence and Frequency of Deployments

In progress

High

07/08/24 - 12/13/24

icfd-increasing-confidence-and-freq-of-deploys

Status Summary

Overall Progress

Details

Targets

Context

A key engineering goal for 2024 is to increase the **confidence** and **frequency** of commit → prod delivery workflows for spinnaker based services with the goal of unlocking the ability to drive more fully automated changes with less effort from teams. As such, [Increasing Confidence and Frequency of Deployments](#) is focused on achieving the following 2 outcomes:

- Deployment frequency of at least weekly (i.e. 1 deploy per week)
- Confidence scores of AGREE or STRONGLY AGREE regarding confidence in accepting changes that sit outside main application code (dependencies, baseOS, updates, sidecars, etc.)

Completion criteria

This campaign assistant will guide you through recommended actions to improve your application's deployment posture. Please implement the ones that make sense for your service and your specific use case.

- The steps are divided into several sections, each covering a relevant aspect of the SDLC.
- Each section starts with an overview of what we are asking for and why – and then proceeds to steps with actions you should review, address, and acknowledge when you've completed them.
- For some use cases, you may need to invest in additional protections not covered by this guide -- please do what is necessary and don't feel limited by this list.

Work involved

Applications must achieve a confidence score when deploying external dependencies of AGREE or STRONGLY AGREE AND

About

Priority	High
Targets owned by your team	5
Launch Date	Jul 8, 2024
Plans Due Date	Aug 15, 2024
Completion Date	Dec 13, 2024
Campaign Owner	
Campaign Team	
Resources	Increasing Confidence and Frequency of Deployments Historical Burndown Status Progress Analytics

(Image from internal Console tool)

We have different types of campaigns



Migration



Incident



Vulnerability

Example of campaign

The screenshot shows the 'Console' interface with a 'Campaigns' section. The main campaign is titled 'JDK 25 for Applications'. It has a status of 'Draft', a priority of 'P4', and a duration of '03/31/26 - 12/31/26'. The contact is 'Slack channel'. The status summary shows a progress bar, and the overall progress is also indicated by a bar. Below the campaign details, there are tabs for 'Details', 'Targets', 'Awaiting Review', 'Change Log', 'Notification Log', and 'Dashboards'.

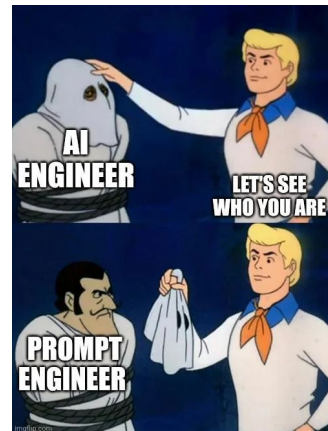
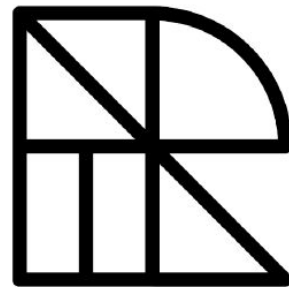
The screenshot shows a GitHub pull request titled 'JDK 21 upgrade for applications #317'. The pull request is from 'change-automat...' and wants to merge 1 commit into 'corp:master' from 'corp:automerge/fleetwide-change-management/jdk21-upg...'. The pull request has 9 conversations, 1 commit, 0 checks, and 3 files changed. A comment from 'change-automation (bot)' dated Aug 8 says 'Use JDK 21 for application modules'. The pull request is approved by 'eb2d985'.

Code Changes are crafted with many tools!



Code transformation tools

- OpenRewrite
- AI prompts
- JetBrains Program Structure Interface (PSI)
- Comby
- And others...



N

but
let's not
forget



NETFLIX

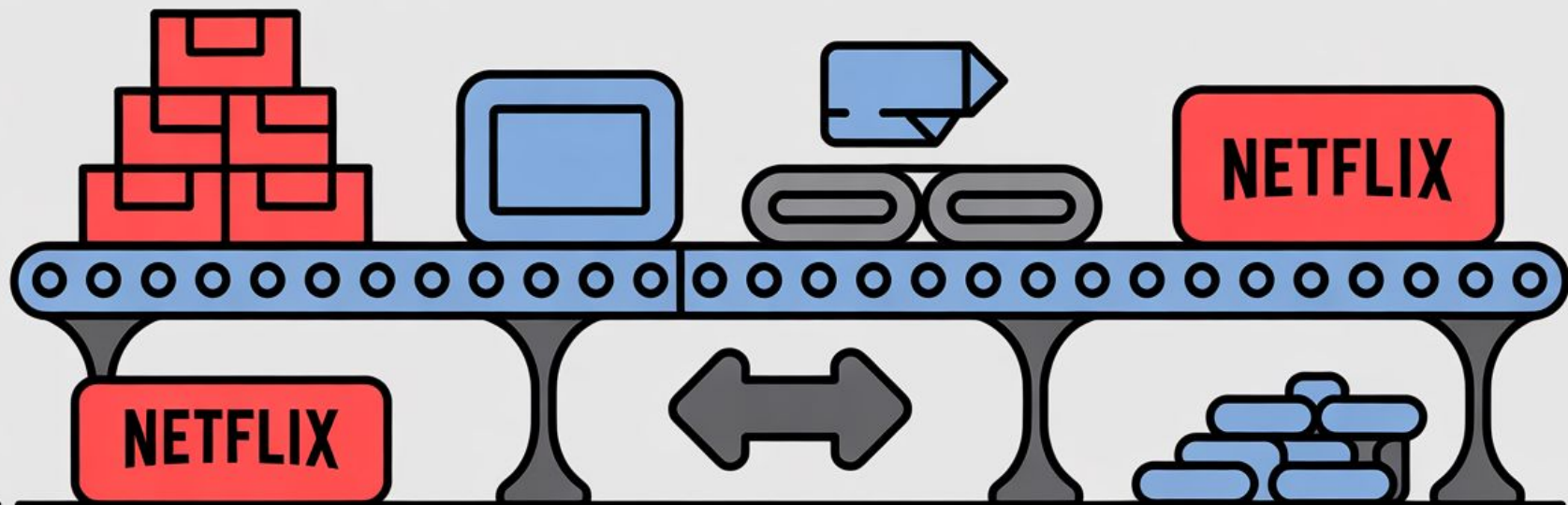
**Merging
Pull Requests
is not**

ENOUGH!



N

CONTINUOUS DELIVERY



N

Streamlined Deployment Process

Infrequent
Manual
Deployments

Slow, error-prone
release process

Increase
Deployment
Frequency

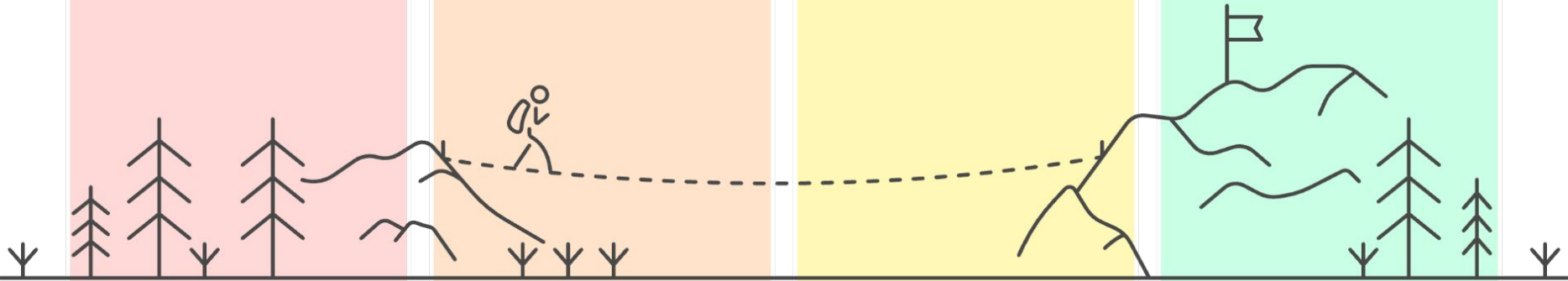
More frequent
releases

Reduce Manual
Verification

Automate checks
and validations

Automated
Production
Deployments

Fast, reliable release
process



NETFLIX

Increased confidence and frequency of Deployments

Image generated with <https://ideogram.ai/>



Why are people not merging and deploying changes?

The image consists of two overlapping screenshots. The background screenshot shows a GitHub 'Pull requests' page with a list of open pull requests. A red bracket on the left side of the list is labeled 'Not merged'. The foreground screenshot shows the 'Environments' section of a Spinnaker deployment tool. Under the 'prod' environment, which is marked 'Not deployed', there is a 'Pending' build (#30) highlighted with a red box. This build is associated with 'PR #12: Merge pull request #12 in ...'. Below the 'Pending' build is a link that says 'Show 10 more pending builds...'. Other builds shown include a 'Live' build (#17) for 'PR #8: Merge pull request #8 in ...' and a 'Resources' section.

Not merged

Environments

prod Not deployed

> Pending #30 PR #12: Merge pull request #12 in ...

Show 10 more pending builds...

> Live #17 PR #8: Merge pull request #8 in ...

> Resources

N

Can we get to
zero-touch
deploys for these
campaigns?



We sent a survey to software developers

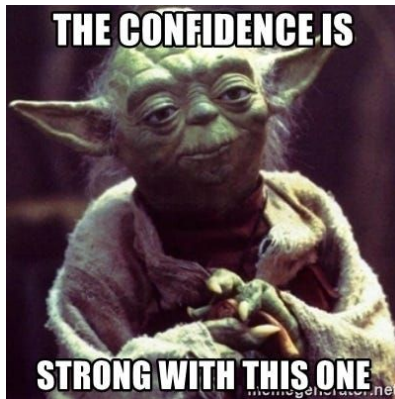


Confidence questions



Automatic commit to deployment workflows with the following types of changes:

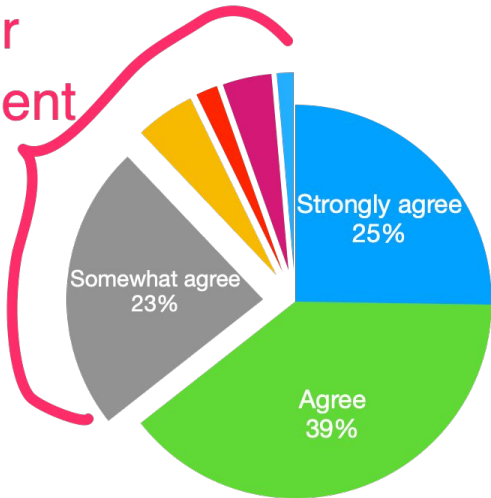
1. Application code changes made by your own team?
2. Application code changes made from engineers outside your team?
3. Changes outside of the application code?



How 2024 looked like

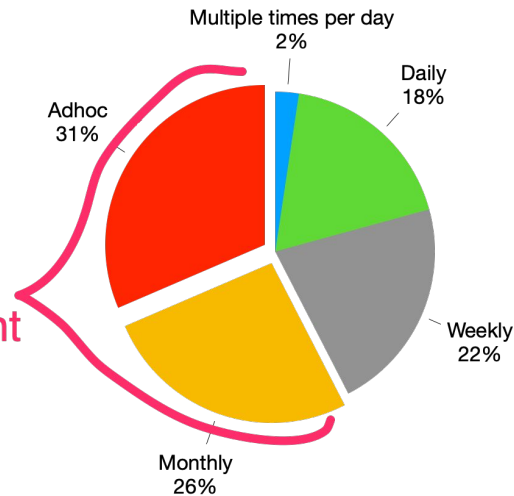
Confidence Rating per app for changes outside of application code

Targets for
improvement



Frequency of Deployments

Targets for
improvement



**Maybe the
paved path
had some
gravel
sections**





Image generated with ChatGPT

Virtual team

26 passionate individuals from:

- Platform Engineering teams
- Product Engineering teams
- Product Management
- Subject Matter Experts

Engineering requirements

Require improvements in different categories:

- Application metadata
- Build and Inner Dev Loop Testing infrastructure
- Delivery infrastructure
- Security
- Source Code quality
- Resiliency and Scalability Testing
- Observability
- And more!



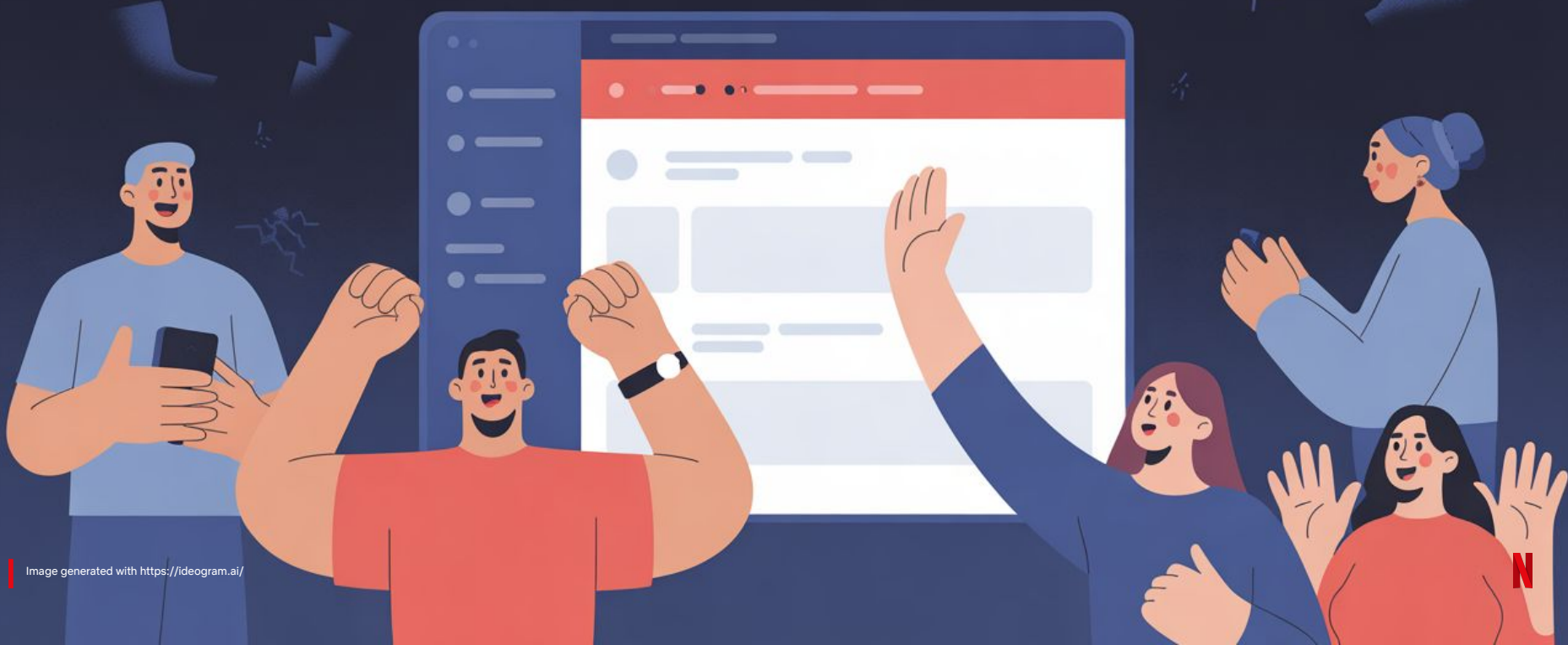
NETFLIX

 Survey



NETFLIX

We prioritized and tackled many of them



1

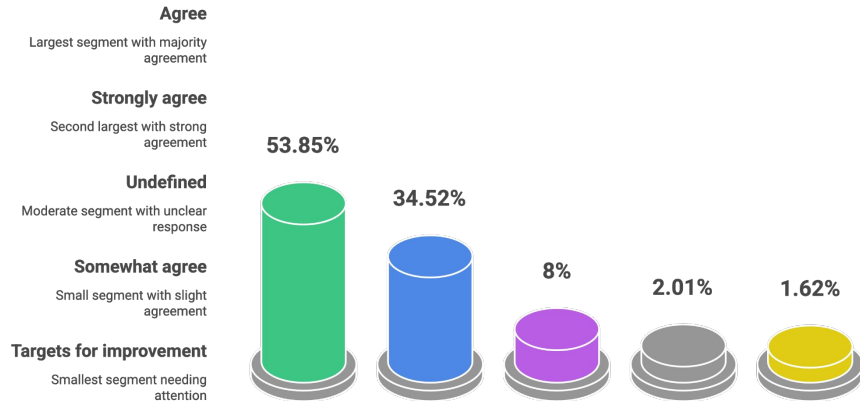
N

(Image from internal Console tool)

We made great progress!

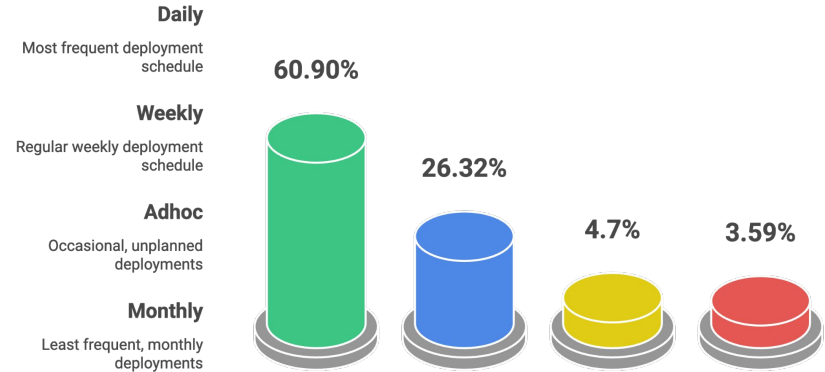
Confidence Rating per app for changes outside of application code

Survey Results on Agreement Levels



Frequency of Deployments

Deployment Frequency



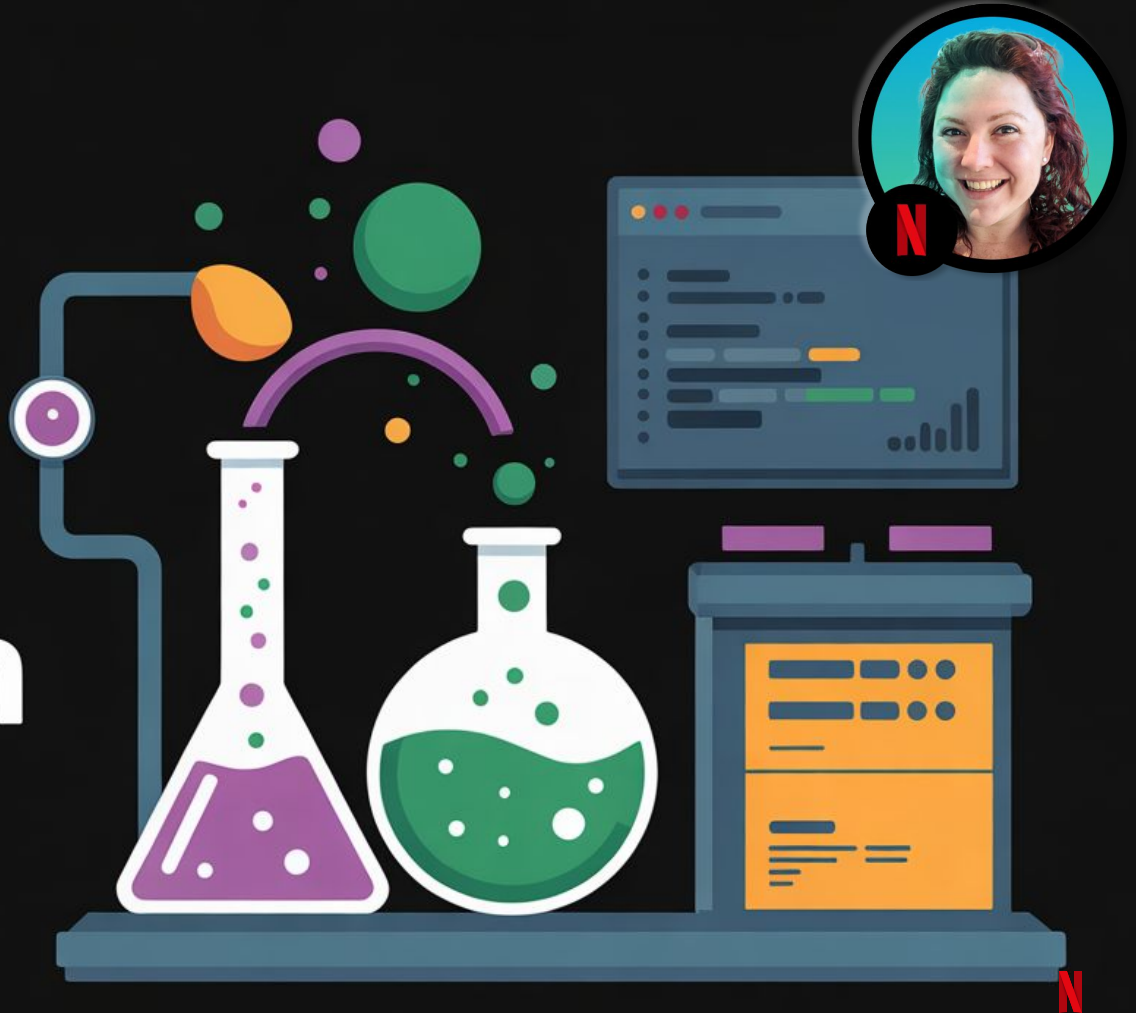
N

Huge Success

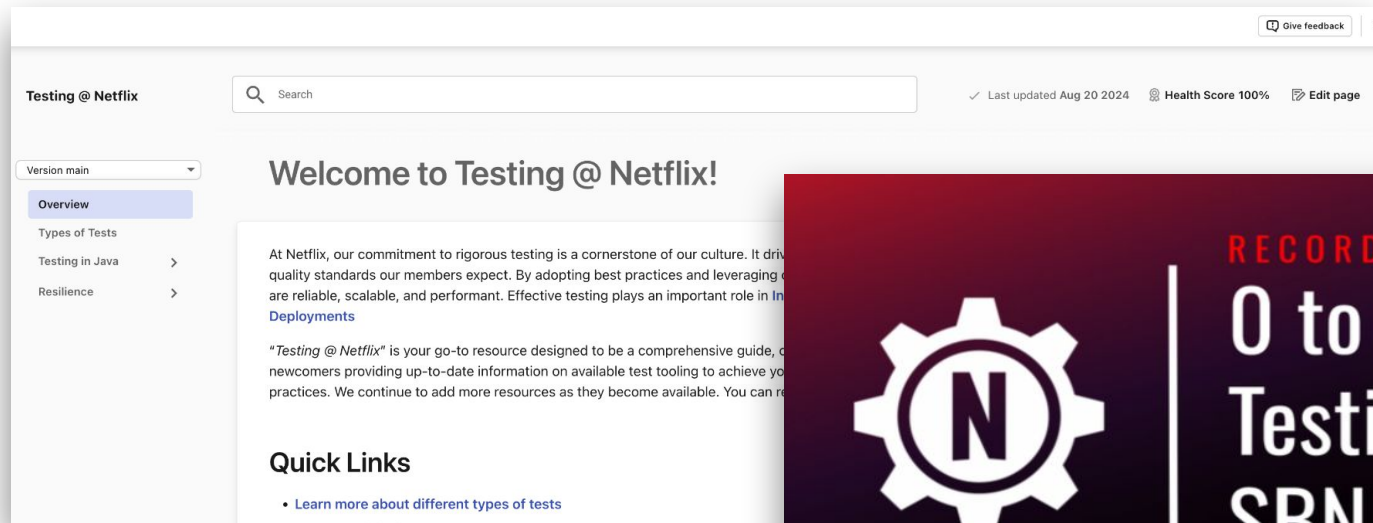


NETFLIX

Testing and Validation



Curated Recommendations



Improving documentation increased our CSAT considerably!

Testing insights



69% Code Coverage

↑ +38%



[Documentation](#)

[Code Coverage Report](#)

93% Test Success Rate

↓ -7%



[Documentation](#)

[Unstable Tests Report](#)

12m 12s Test Execution Time

↑ +5345%



[Documentation](#)

[Slowest Tests Report](#)

Testing insights



Data with organization rollup-views

(1) Test Insights

Beta

Search names

Lifecycle state

All Software Lifecycles

Directly Owned Resources

All Resources

Name	Code coverage	90-d... $\equiv \uparrow$	Test success rate	90-day Δ	Test execution time	90-day Δ	# of tests	90-day Δ
 	15%	$\uparrow +5\%$	100%	$\pm 0\%$	43s	$\uparrow +15\%$	2	$\pm 0\%$
 	90%	$\pm 0\%$	100%	$\pm 0\%$	3m 28s	$\uparrow +53\%$	377	$\uparrow +2\%$
 	79%	$\pm 0\%$	100%	$\pm 0\%$	3m 48s	$\uparrow +12\%$	9	$\pm 0\%$
 	27%	$\pm 0\%$	100%	$\pm 0\%$	37s	$\pm 0\%$	1	$\pm 0\%$
 	83%	$\pm 0\%$	100%	$\pm 0\%$	1m 23s	$\uparrow +5\%$	76	$\pm 0\%$

Testing Scorecards



Images from internal UI backed by internal data

Console

Search

Manage

Discover

Platform

?

Service

Overview

Scorecards

Testing for [redacted]

Testing Gold

Get insights into your testing health with rules evaluating code coverage and 30-day default-branch test suite success rates.

Visit [https://\[redacted\]/testing](https://[redacted]/testing) to learn testing best practices ...

Expand

Gold

100%

Passing rules for all levels (6)

- > 30-day test suite success rate > 90%
- > 30-day test suite success rate > 94%
- > 30-day test suite success rate > 97%
- > Test coverage > 25%
- > Test coverage > 50%
- > Test coverage > 75%

Level



Delivery notifications



APP 9:30 AM

Deployment Frequency Summary for 10 applications (10 applications)



— 8 days ago (2 pending artifacts)



— | | |
| | | |
| |



— Deployed to production within last 14 day(s).



— Deployed to production within last 7 day(s).

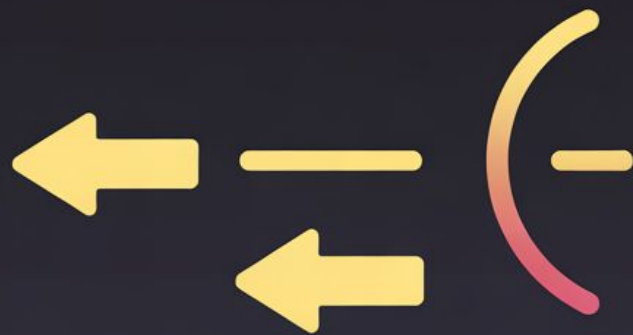


3 replies Last reply today at 9:31 AM

Delivery Score Cards

Metadata	Scores		🏆 Bronze	🥈 Silver	🥇 Gold
	Score	Score Percentage	Monthly deployments	Weekly Deployments	Daily Deployments or More
Service        	1	33%	1	0	0
	3	100%	1	1	1
	3	100%	1	1	1
	3	100%	1	1	1
	3	100%	1	1	1
	3	100%	1	1	1
	2	67%	1	1	0

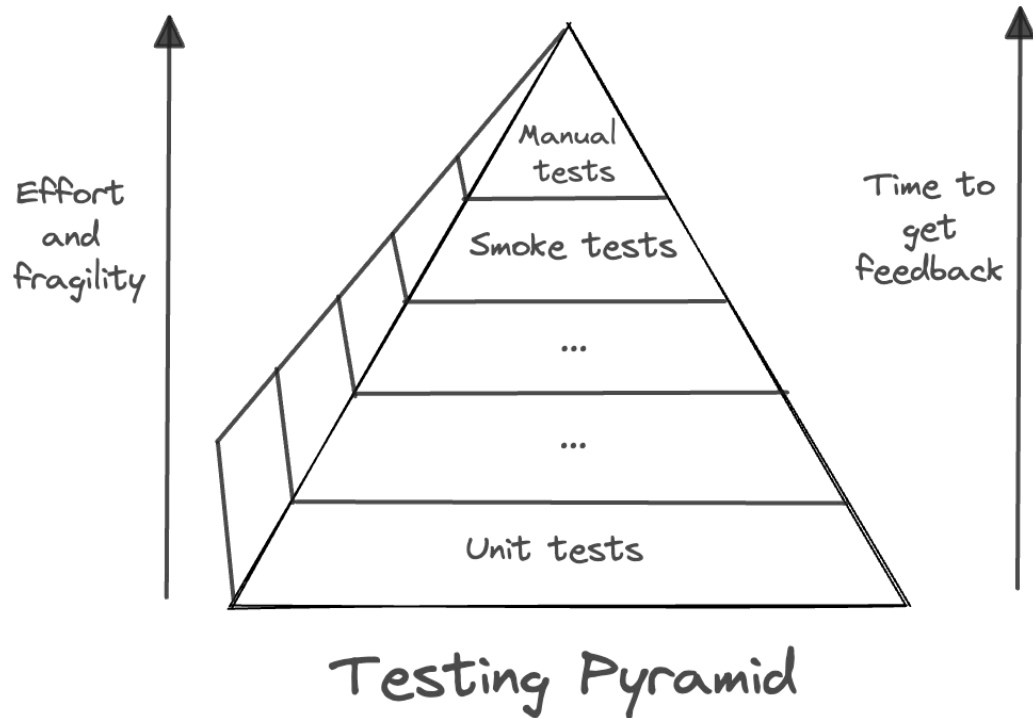
NETFLIX



Shift
Left
Testing



Why is this important?

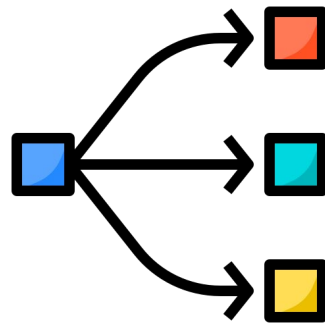


Remote Test Execution



Take existing test suites and distribute them across remote agents to execute them faster

The tests and their supporting files are transferred to each agent and executed, with their logging and results streamed back to the build in real time.



 **DEV**/ELOCITY

Predictive Test Selection

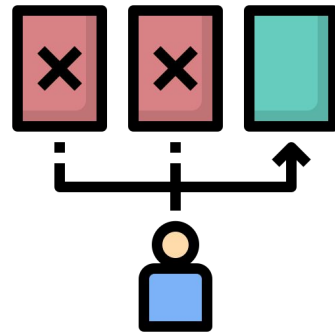
Uses a predictive model by observing code changes and test outcomes from your Build Scan data



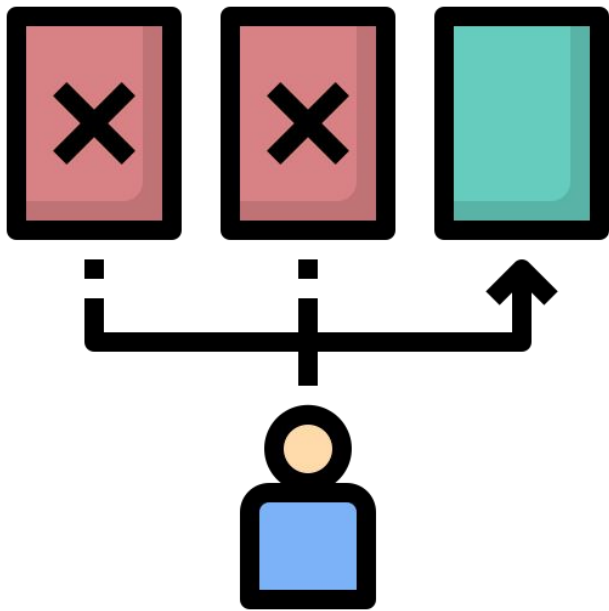
 **DEV**/ELOCITY

Flaky Test Detection

When a test fails, how do we determine if it's flaky or not?



Flaky Test Visualization



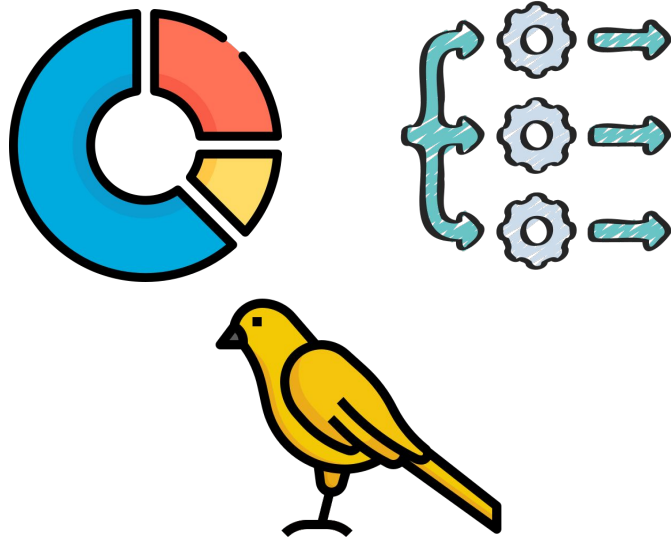
We use Test Retry Gradle plugin with Develocity to catch and visualize flakiness.



Developer Experience: Testing, *continued*

1. Test slices
2. JUnit Platform test parallelization
3. Improving canaries

And many more!



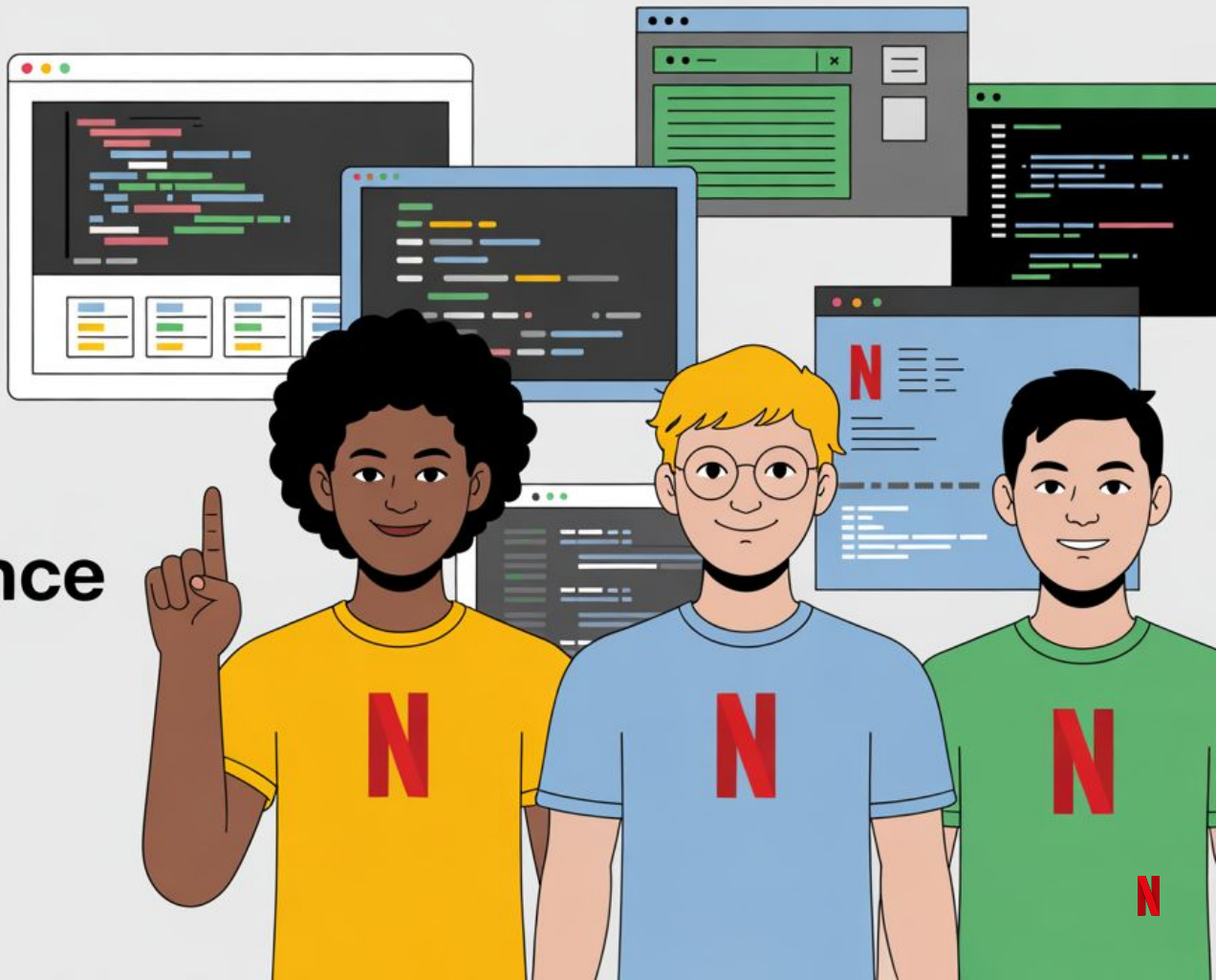
NETFLIX

Everything is perfect and our job is done, right?



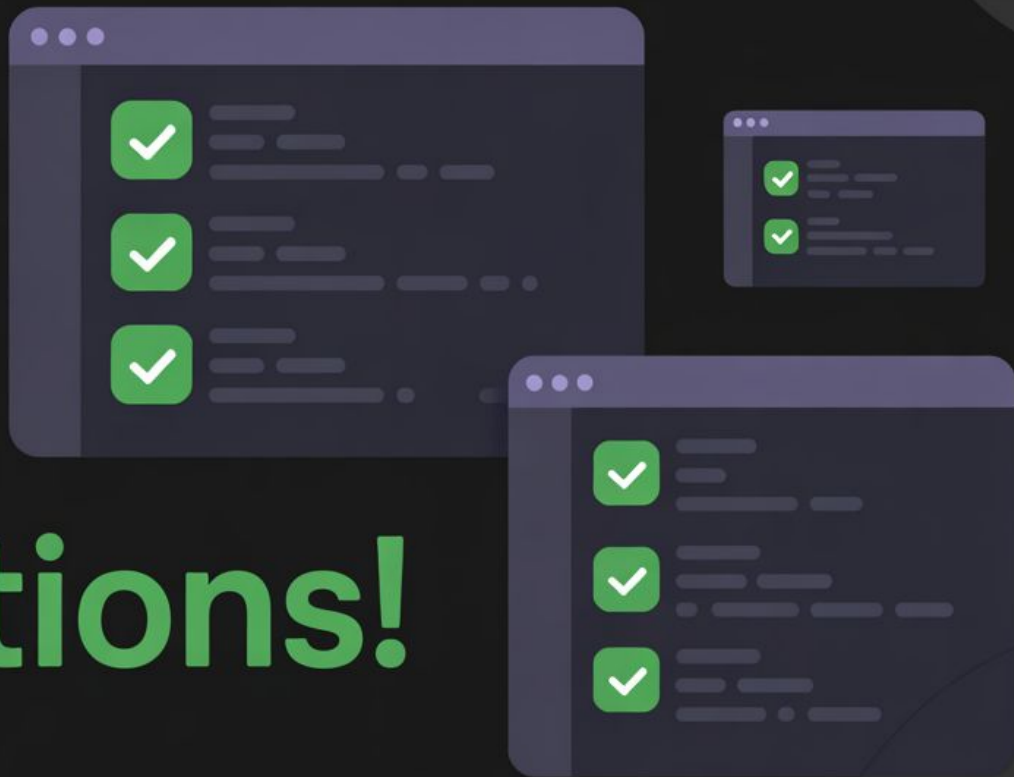
N

It is **NOT**,
and we keep
investing
in this experience



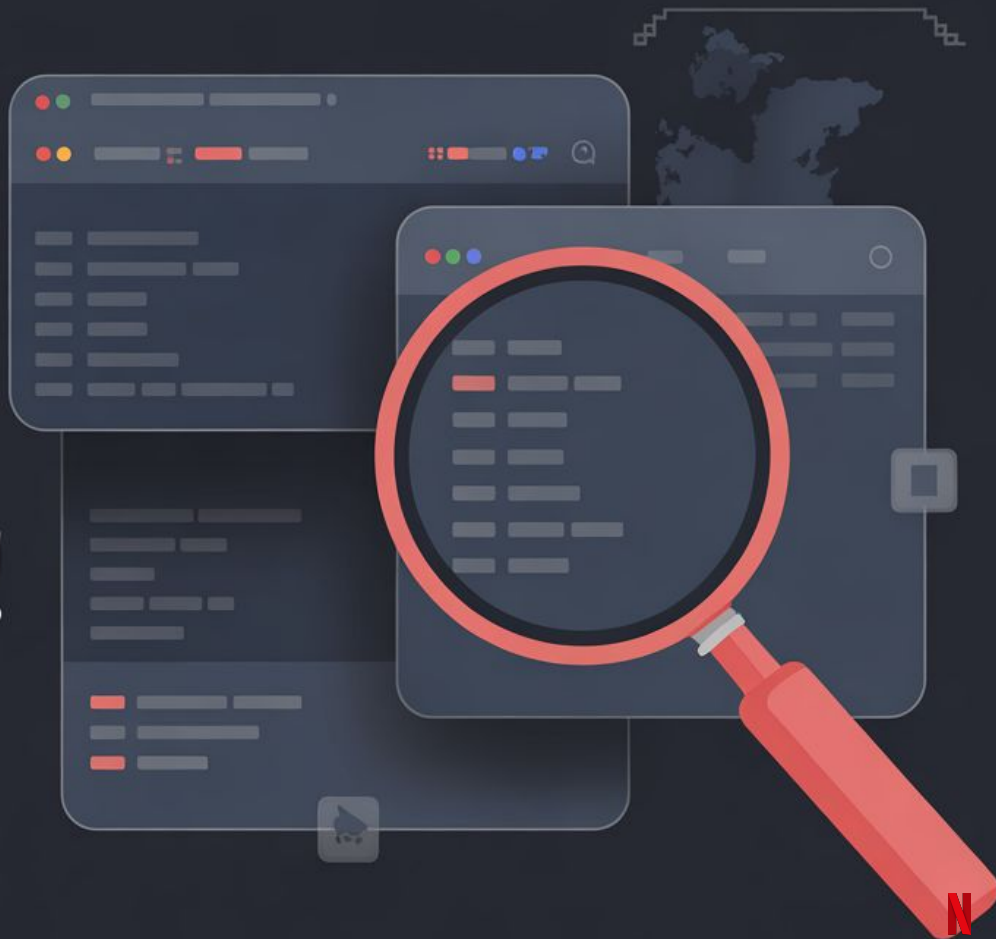
NETFLIX

More
Validations!



NETFLIX

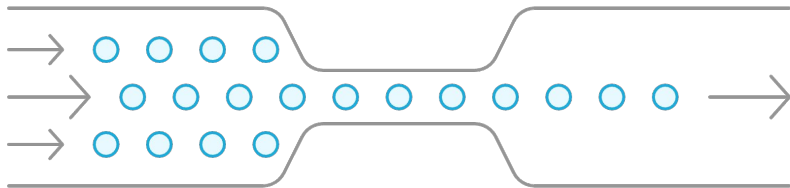
Better
artifact
observability!



LET'S N
RECAP

Let's recap

Developers struggle to keep up with frequent updates.



Foundations of Delivering Changes at Scale

Build Opinions



Distributed Refactoring



Testing and Validation



Artifact
Observability



Continuous
Delivery

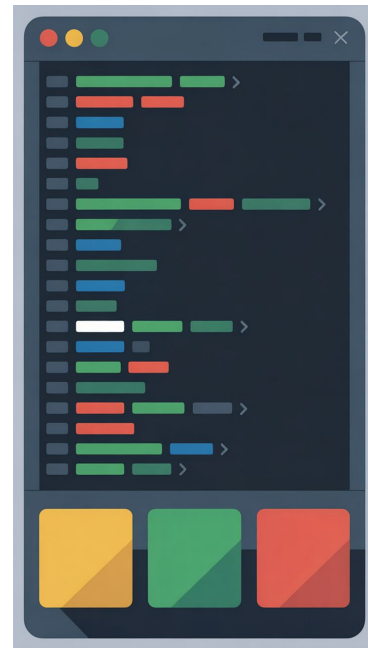
Takeaways

Code changes are a part of life.

We must be adaptable and flexible to these changes.

And why do we care about doing this well?

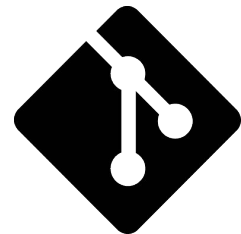
Doing this badly costs toil, friction, and money



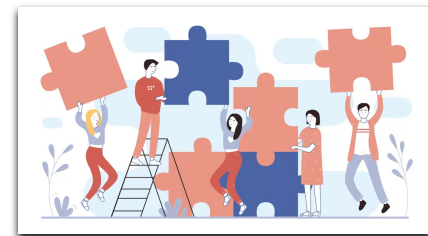
2

Takeaways

Implementing code changes
goes beyond
transformations
and
scheduling.



*There are great companies
solving and tackling this aspect,
though*



3

Takeaways

Making and releasing
fully automated code changes
within the organization
has to be a priority with a
cultural
focus.



Image generated with
<https://ideogram.ai/>

Questions?



Roberto Perez Alcolea
rperezalcolea@netflix.com

Aubrey Chipman
achipman@netflix.com

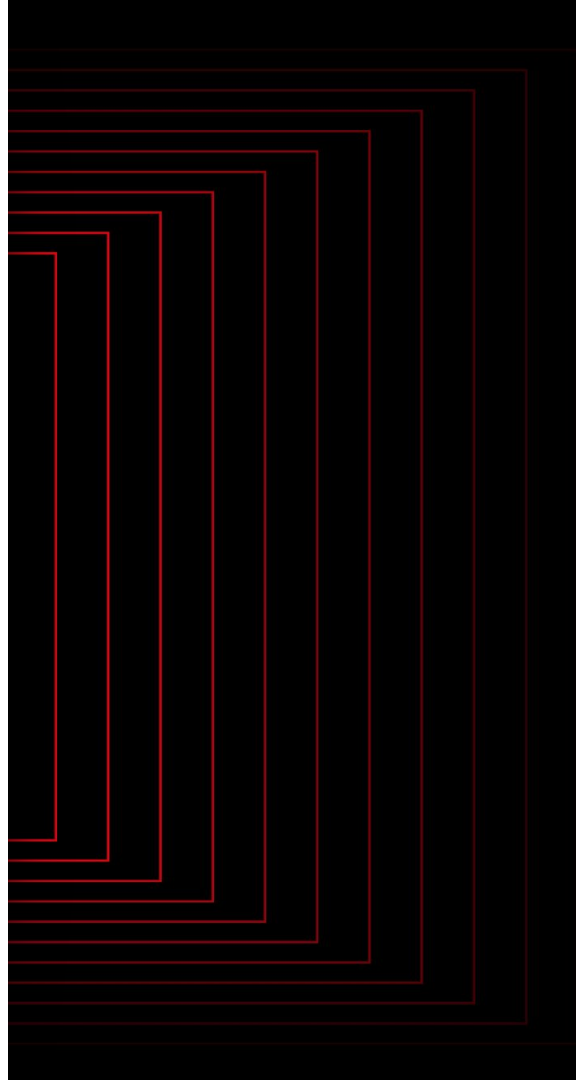


Thank you!



Roberto Perez Alcolea
rperezalcolea@netflix.com

Aubrey Chipman
achipman@netflix.com



Post-presentation update

We will try to address questions submitted during our talk in this doc in the upcoming days!

Check it out here:

